

Soil Fw doc

v1.0.0

Generated by Doxygen 1.9.1

Chapter 1

Sample project

(See the README.md file in the upper level 'examples' directory for more information about examples.)

This is the simplest buildable example. The example is used by command `idf.py create-project` that copies the project to user specified path and set it's name. For more information follow the [docs page](#)

1.1 How to use example

We encourage the users to use the example as a template for the new projects. A recommended way is to follow the instructions on a [docs page](#).

1.2 Example folder contents

The project **sample_project** contains one source file in C language `main.c`. The file is located in folder `[main](main)`.

ESP-IDF projects are built using CMake. The project build configuration is contained in `CMakeLists.txt` files that provide set of directives and instructions describing the project's source files and targets (executable, library, or both).

Below is short explanation of remaining files in the project folder.

```
CMakeLists.txt
main
  CMakeLists.txt
  main.c
README.md           This is the file you are currently reading
```

Additionally, the sample project contains Makefile and component.mk files, used for the legacy Make based build system. They are not used or needed when building with CMake and idf.py.

1.3 ESP IDF

Esp idf version `v4.4.1` eclipse [plugin](#)

1.4 Compilation

```
rm -rf build
get_idf
idf.py set-target esp32s3
idf.py all
```

1.5 Flash

```
idf.py flash monitor
```

Chapter 2

Deprecated List

Global [app_call_states](#)

Chapter 3

Module Index

3.1 Modules

Here is a list of all modules:

Main	??
Comm_app	??

Chapter 4

Data Structure Index

4.1 Data Structures

Here are the data structures with brief descriptions:

async_resp_arg		??
comm_app_request		??
http_file_server_data		??
i2c_dev_t		
Structure representing an I2C device		??
pivot_sectors		
Configuration angles		??

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

build/bootloader/config/ sdkconfig.h	??
build/config/ sdkconfig.h	??
build/esp-idf/mbdtdls/mbdtdls/library/ ssl_debug_helpers_generated.c	
Automatically generated helper functions for debugging	??
components/common_communication/include/ common_parser.h	
Parse and convert incoming and outgoing messages	??
components/gpio_actuator/ gpio_actuator.c	
Implementation of GPIO actuator control functions	??
components/gpio_actuator/include/ gpio_actuator.h	
Board GPIO macros and pinout definitions for the GPIO Actuator module	??
components/gprs/ gprs_module.c	
Controls the flow of reception and transmission of the GPRS module	??
components/gprs/include/ gprs_module.h	
GPRS Module API	??
components/gprs/include/ gprs_uart.h	
Header file for GPRS UART communication functions	??
components/http_server/include/ http_api.h	??
components/http_server/include/ http_config_parser.h	??
components/http_server/include/ http_storage.h	??
components/idp_protocol/include/ idp_parser.h	??
components/nvs_data/ nvs_data.c	
Class for direct access to flash memory (NVS)	??
components/nvs_data/include/ nvs_data.h	
Class for direct access to flash memory (NVS)	??
components/rf_module/ rf_module.c	
Controls the flow of reception and transmission of the RF module	??
components/rf_module/include/ rf_module.h	??
components/rf_module/include/ rf_uart.h	??
components/rtc/ rtc_app.c	
RTC application implementation for managing date and time functionality	??
components/rtc/ rtc_ds3231.c	
RTC DS3231 class implementation	??
components/rtc/ rtc_i2c_dev.c	
RTC I2C device implementation	??
components/rtc/include/ rtc_app.h	
RTC application class for managing date and time functionality	??

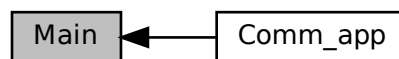
components/rtc/include/ rtc_ds3231.h	
RTC DS3231 class for interfacing with the DS3231 RTC module	??
components/rtc/include/ rtc_i2c_dev.h	??
components/utils/include/ FreeRTOS_defines.h	
Defines for FreeRTOS tasks and priorities	??
components/utils/include/ log.h	
Creating customizable logs	??
components/utils/include/ project_config.h	
Configuration and parameter definitions for the project	??
components/wifi_app/ wifi_app.c	
Wi-Fi application implementation	??
components/wifi_app/include/ wifi_app.h	
Wi-Fi application functions and callbacks	??
main/ actuation_app.c	
Actuation control class	??
main/ comm_app.c	
Communication class control application	??
main/ data_app.c	
Memory control application	??
main/include/ actuation_app.h	
Actuation control class	??
main/include/ comm_app.h	??
main/include/ data_app.h	
Memory control application	??
main/include/ system_manager.h	??

Chapter 6

Module Documentation

6.1 Main

Collaboration diagram for Main:



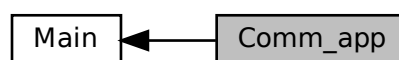
Modules

- [Comm_app](#)

6.1.1 Detailed Description

6.2 Comm_app

Collaboration diagram for Comm_app:



Data Structures

- struct [comm_app_request](#)

Macros

- #define **COMM_APP_TAG** "comm_app"
- #define **COMM_APP_SIZE_QUEUE** 10

Functions

- void [comm_app_task](#) (void *arg)
communication reception task
- bool [comm_app_init](#) (const [app_callback](#) callback)
starts all communication modules
- uint16_t [comm_app_get_degree](#) (void)
starts all communication modules
- void [comm_app_send_event](#) (pivot_actions pivot_status)
send events over communication interfaces
- void **comm_app_set_config** (const pivot_config config)
- void **comm_app_set_actions** (const pivot_actions action, const pivot_config config, uint16_t start_angle, uint16_t end_angle)
- void **comm_app_send_actions** (void)
- void **comm_app_send_idp_resp** ([idp_type](#) idp, char *pivo_id, char *scheduling_id)

6.2.1 Detailed Description

6.2.2 Function Documentation

6.2.2.1 [comm_app_get_degree\(\)](#)

```
uint16_t comm_app_get_degree (
    void )
```

starts all communication modules

Returns

angle: returns the current angle (degrees) received

Here is the call graph for this function:



6.2.2.2 comm_app_init()

```
bool comm_app_init (
    const app_callback callback )
```

starts all communication modules

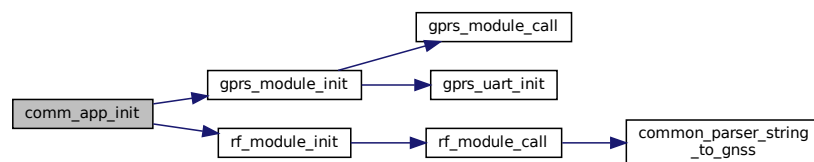
Parameters

<code>app_callback[in]</code>	function used with return to main application class
-------------------------------	---

Returns

- true: success
- false: fail

Here is the call graph for this function:



6.2.2.3 comm_app_send_event()

```
void comm_app_send_event (
    pivot_actions pivot_status )
```

send events over communication interfaces

Parameters

<code>pivot_status[in]</code>	configuration structure to send
-------------------------------	---------------------------------

6.2.2.4 comm_app_task()

```
void comm_app_task (
    void * arg )
```

communication reception task

Parameters

<i>arg[in]</i>	: task argument (default NULL)
----------------	--------------------------------

Chapter 7

Data Structure Documentation

7.1 `async_resp_arg` Struct Reference

Data Fields

- `httpd_handle_t` `hd`
- `int` `fd`

The documentation for this struct was generated from the following file:

- `components/http_server/http_api.c`

7.2 `comm_app_request` Struct Reference

Data Fields

- `idp_type` `request_idp`
- `char` `request_buffer` [100]

7.2.1 Detailed Description

Structure sent to the control queue

7.2.2 Field Documentation

7.2.2.1 `request_buffer`

```
char request_buffer[100]
```

Configuration input

7.2.2.2 request_idp

`idp_type request_idp`

Request types sent to the control queue

The documentation for this struct was generated from the following file:

- `main/comm_app.c`

7.3 http_file_server_data Struct Reference

Data Fields

- `char base_path [ESP_VFS_PATH_MAX+1]`
- `char scratch [HTTP_SCRATCH_BUFSIZE]`

7.3.1 Detailed Description

data structure for HTTP transmission

The documentation for this struct was generated from the following file:

- `components/http_server/http_api.c`

7.4 i2c_dev_t Struct Reference

Structure representing an I2C device.

```
#include <rtc_i2c_dev.h>
```

Data Fields

- `i2c_port_t port`
- `uint8_t addr`
- `gpio_num_t sda_io_num`
- `gpio_num_t scl_io_num`
- `uint32_t clk_speed`

7.4.1 Detailed Description

Structure representing an I2C device.

The documentation for this struct was generated from the following file:

- `components/rtc/include/rtc_i2c_dev.h`

7.5 pivot_sectors Struct Reference

Configuration angles.

```
#include <project_config.h>
```

Data Fields

- `uint8_t id`
- `uint16_t start_angle`
- `uint16_t end_angle`

7.5.1 Detailed Description

Configuration angles.

Structure defining the configuration angles.

7.5.2 Field Documentation

7.5.2.1 end_angle

```
uint16_t end_angle
```

End angle of the configuration

7.5.2.2 id

```
uint8_t id
```

ID of the configuration angle

7.5.2.3 start_angle

```
uint16_t start_angle
```

Start angle of the configuration

The documentation for this struct was generated from the following file:

- `components/utils/include/project_config.h`

Chapter 8

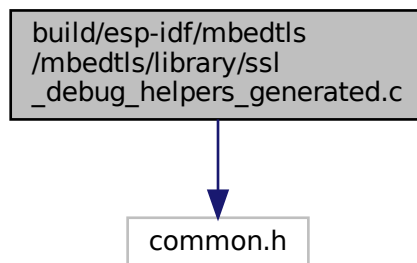
File Documentation

8.1 build/esp-idf/mbdttl/mbdttl/library/ssl_debug_helpers_↵ generated.c File Reference

Automatically generated helper functions for debugging.

```
#include "common.h"
```

Include dependency graph for ssl_debug_helpers_generated.c:



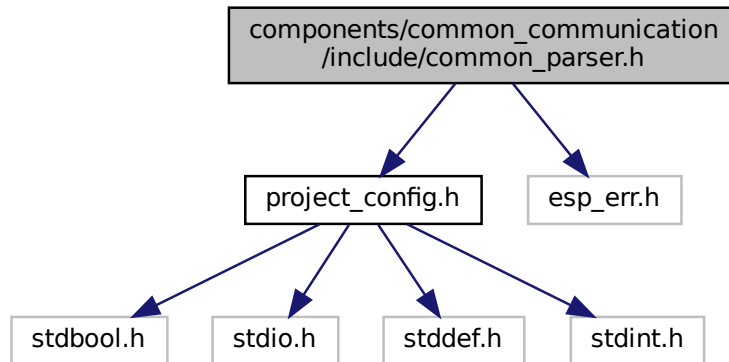
8.1.1 Detailed Description

Automatically generated helper functions for debugging.

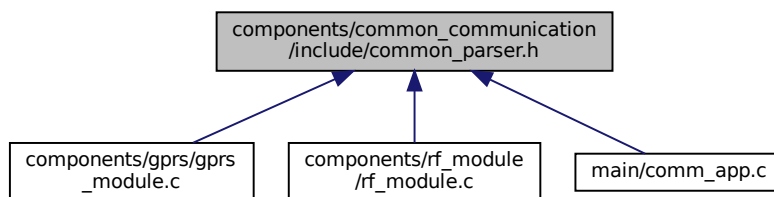
8.2 components/common_communication/include/common_parser.h File Reference

parse and convert incoming and outgoing messages

```
#include "project_config.h"
#include "esp_err.h"
Include dependency graph for common_parser.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t common_parser_string_to_action` (const char *string_in, pivot_actions *action_out)
convert a string to a configuration structure
- `esp_err_t common_parser_string_to_gnss` (const char *string_in, uint16_t *angle, time_t *timestamp)
convert string to angle and timestamp variables
- `esp_err_t common_parser_status_to_string` (pivot_actions config_in, time_t timestamp, uint16_t angle, char *string_out)
convert configuration structure to string
- `esp_err_t common_parser_json_to_timestamp` (const char *string_in, time_t *timestamp)
- `esp_err_t common_parser_status_to_json` (pivot_actions config_in, time_t timestamp, uint16_t angle, const char *id, char *string_out, size_t string_size)
- `esp_err_t common_parser_register_to_json` (const char *id, char *string_out, size_t string_size)
- `idp_type common_parser_get_idp` (const char *string_in)
- `esp_err_t common_parser_string_to_scheduling_date` (char *string_in, pivot_scheduling_date *action_out)
convert a string to a configuration structure

- `esp_err_t common_parser_string_to_scheduling_angle` (`char *string_in`, `pivot_scheduling_angle *scheduling_out`)
- `esp_err_t common_parser_ipm_resp` (`idp_type idp`, `char *pivo_id`, `char *scheduling_id`, `char *resp_out`)
- `esp_err_t common_parser_scheduling_date_http_to_mqtt` (`idp_type idp`, `char *gprs_id`, `pivot_↵ scheduling_date scheduling_in`, `char *string_out`)
- `esp_err_t common_parser_scheduling_angle_http_to_mqtt` (`idp_type idp`, `char *gprs_id`, `pivot_↵ scheduling_angle scheduling_in`, `char *string_out`)

8.2.1 Detailed Description

parse and convert incoming and outgoing messages

communication class control application

Date

June 21, 2022

June 23, 2022

8.2.2 Function Documentation

8.2.2.1 common_parser_status_to_string()

```
esp_err_t common_parser_status_to_string (
    pivot_actions config_in,
    time_t timestamp,
    uint16_t angle,
    char * string_out )
```

convert configuration structure to string

Parameters

<i>config_in[in]</i>	input configuration structure
<i>timestamp[out]</i>	input timestamp
<i>angle[out]</i>	angle in degrees
<i>string_out[out]</i>	converted string

Returns

- ESP_OK: success
- ESP_FAIL: fail

8.2.2.2 common_parser_string_to_action()

```
esp_err_t common_parser_string_to_action (
    const char * string_in,
    pivot_actions * action_out )
```

convert a string to a configuration structure

Parameters

<i>sting_in</i> [in]	input string, format(xxx-xxx)
<i>config_out</i> [out]	output configuration structure

Returns

- ESP_OK: success
- ESP_FAIL: fail

8.2.2.3 common_parser_string_to_gnss()

```
esp_err_t common_parser_string_to_gnss (
    const char * string_in,
    uint16_t * angle,
    time_t * timestamp )
```

convert string to angle and timestamp variables

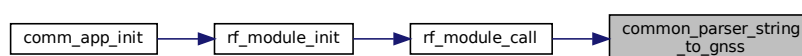
Parameters

<i>sting_in</i> [in]	input string, format(GPS...)
<i>angle</i> [out]	angle in degrees converted
<i>timestamp</i> [out]	converted timestamp

Returns

- ESP_OK: success
- ESP_FAIL: fail

Here is the caller graph for this function:



8.2.2.4 common_parser_string_to_scheduling_date()

```
esp_err_t common_parser_string_to_scheduling_date (
    char * string_in,
    pivot_scheduling_date * action_out )
```

convert a string to a configuration structure

Parameters

<i>string_in[in]</i>	input string, format(xxx-xxx)
<i>config_out[out]</i>	output configuration structure

Returns

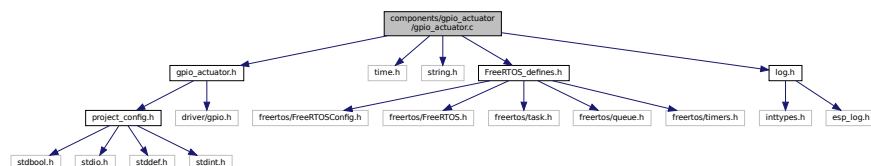
- ESP_OK: success
- ESP_FAIL: fail

8.3 components/gpio_actuator/gpio_actuator.c File Reference

Implementation of GPIO actuator control functions.

```
#include "gpio_actuator.h"
#include <time.h>
#include <string.h>
#include "FreeRTOS_defines.h"
#include "log.h"
```

Include dependency graph for gpio_actuator.c:



Macros

- `#define GPIO_ACT_TAG "gpio_actuator"`
Tag used for logging in the GPIO actuator module.
- `#define GPIO_ACT_INTR_FLAG_DEFAULT 0`
Default interrupt flag for GPIO handling.

Functions

- void [vPercTimerOnExpire](#) (TimerHandle_t pxTimer)
Callback function for the expiration of the Perc On timer.
- void [vPercTimerOffExpire](#) (TimerHandle_t pxTimer)
Callback function for the expiration of the Perc Off timer.
- esp_err_t [gpio_actuator_start](#) (void)
Start the relays accordingly after pressure check or dry mode.
- void [actuator_wait_pressure](#) (void *arg)
Water pressure application task.
- void [actuator_read_percent](#) (void *arg)
Percentimeter reading task.
- esp_err_t [gpio_actuator_init](#) ()
Initializes the GPIO actuator module.
- esp_err_t [gpio_actuator_set](#) (pivot_actions config)
Sets the pivot configuration.
- pivot_actions [gpio_actuator_get](#) (void)
Gets the current pivot configuration.
- void [gpio_actuator_shutdown](#) (void)
Shuts down all relays.
- void [gpio_actuator_pump_on](#) (void)
Turns on the pump relay.
- void [gpio_actuator_pump_off](#) (void)
Turns off the pump relay.
- void [gpio_actuator_pressure_on](#) (void)
Turns on the pressure relay and starts pressure monitoring.
- void [gpio_actuator_pressure_off](#) (void)
Turns off the pressure relay and stops pressure monitoring.

8.3.1 Detailed Description

Implementation of GPIO actuator control functions.

Date

June 20, 2022

8.3.2 Function Documentation

8.3.2.1 [actuator_read_percent\(\)](#)

```
void actuator_read_percent (
    void * arg )
```

Percentimeter reading task.

Parameters

<i>arg</i>	Task argument (default NULL)
------------	------------------------------

8.3.2.2 actuator_wait_pressure()

```
void actuator_wait_pressure (
    void * arg )
```

Water pressure application task.

Parameters

<i>arg</i>	Task argument (default NULL)
------------	------------------------------

8.3.2.3 gpio_actuator_get()

```
pivot_actions gpio_actuator_get (
    void )
```

Gets the current pivot configuration.

Returns

The current pivot configuration

8.3.2.4 gpio_actuator_init()

```
esp_err_t gpio_actuator_init (
    void )
```

Initializes the GPIO actuator module.

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to initialize

Here is the caller graph for this function:

**8.3.2.5 gpio_actuator_set()**

```
esp_err_t gpio_actuator_set (
    pivot_actions config )
```

Sets the pivot configuration.

Parameters

<i>config</i>	The pivot configuration to set
---------------	--------------------------------

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to set the configuration

8.3.2.6 gpio_actuator_start()

```
esp_err_t gpio_actuator_start (
    void )
```

Start the relays accordingly after pressure check or dry mode.

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to initialize

8.3.2.7 vPercTimerOffExpire()

```
void vPercTimerOffExpire (
    TimerHandle_t pxTimer )
```

Callback function for the expiration of the Perc Off timer.

Parameters

<i>pxTimer</i>	The timer handle
----------------	------------------

8.3.2.8 vPercTimerOnExpire()

```
void vPercTimerOnExpire (
    TimerHandle_t pxTimer )
```

Callback function for the expiration of the Perc On timer.

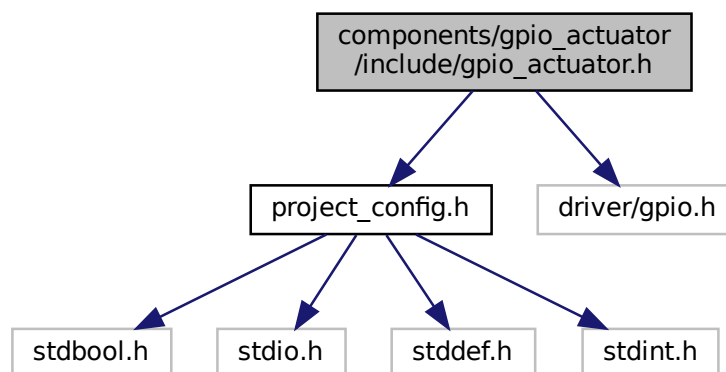
Parameters

<i>pxTimer</i>	The timer handle
----------------	------------------

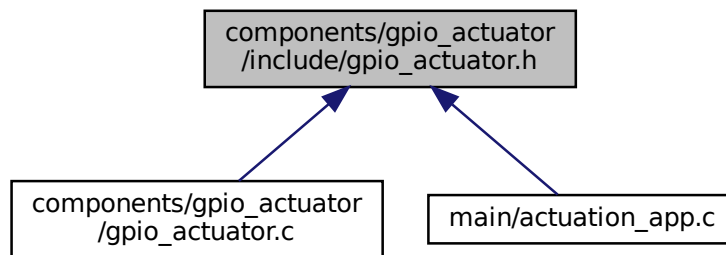
8.4 components/gpio_actuator/include/gpio_actuator.h File Reference

Board GPIO macros and pinout definitions for the GPIO Actuator module.

```
#include "project_config.h"
#include "driver/gpio.h"
Include dependency graph for gpio_actuator.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define GPIO_ACT_SYS_ENABLE 0`
Macro to enable the GPIO actuator system.
- `#define GPIO_ACT_SYS_DISABLE 1`
Macro to disable the GPIO actuator system.
- `#define GPIO_ACT_PERC_FULL_CYCLE 60000`
Macro defining the full cycle duration for the percentimeter (in milliseconds).
- `#define GPIO_ACT_PRESSURE_TIMEOUT 300000`
Macro defining the timeout duration for water pressure reading (in milliseconds).
- `#define GPIO_ACT_ONOFF_DELAY 2000`
Macro defining the delay duration for turning on and off relays (in milliseconds).
- `#define GPIO_ACT_PIN_ON GPIO_NUM_13`
GPIO pin number for turning the system on.
- `#define GPIO_ACT_PIN_OFF GPIO_NUM_12`
GPIO pin number for turning the system off.
- `#define GPIO_ACT_PIN_CW GPIO_NUM_11`
GPIO pin number for clockwise direction.
- `#define GPIO_ACT_PIN_CCW GPIO_NUM_10`
GPIO pin number for counter-clockwise direction.
- `#define GPIO_ACT_PIN_WATERING GPIO_NUM_9`
GPIO pin number for watering relay.
- `#define GPIO_ACT_PIN_AUX GPIO_NUM_8`
GPIO pin number for auxiliary system relay.
- `#define GPIO_ACT_PIN_PERC_AUX GPIO_NUM_16`
GPIO pin number for auxiliary relay for percentimeter.
- `#define GPIO_ACT_PIN_PERC_OUT GPIO_NUM_35`
GPIO pin number for percentimeter reading output.
- `#define GPIO_ACT_PIN_PUMP GPIO_NUM_14`
GPIO pin number for pump relay.
- `#define GPIO_ACT_PIN_CW_IN GPIO_NUM_7`
GPIO pin number for clockwise input.
- `#define GPIO_ACT_PIN_CCW_IN GPIO_NUM_6`
GPIO pin number for counter-clockwise input.

- #define `GPIO_ACT_PIN_PERC_IN` `GPIO_NUM_5`
GPIO pin number for percentimeter read input.
- #define `GPIO_ACT_PIN_PRESS` `GPIO_NUM_4`
GPIO pin number for water pressure reading input.
- #define `GPIO_OUTPUT_PIN_GROUP`
GPIO pin bitmask for all output pins.
- #define `GPIO_INPUT_PIN_GROUP`
GPIO pin bitmask for all input pins.
- #define `GPIO_INT_PERC` `(1ULL<<GPIO_ACT_PIN_PERC_IN)`
GPIO pin bitmask for the percentimeter interrupt pin.

Functions

- `esp_err_t gpio_actuator_init` (void)
Initializes the GPIO actuator module.
- `esp_err_t gpio_actuator_set` (pivot_actions config)
Sets the pivot configuration.
- `pivot_actions gpio_actuator_get` (void)
Gets the current pivot configuration.
- `void gpio_actuator_shutdown` (void)
Shuts down all relays.
- `void gpio_actuator_pump_on` (void)
Turns on the pump relay.
- `void gpio_actuator_pump_off` (void)
Turns off the pump relay.
- `void gpio_actuator_pressure_on` (void)
Turns on the pressure relay and starts pressure monitoring.
- `void gpio_actuator_pressure_off` (void)
Turns off the pressure relay and stops pressure monitoring.

8.4.1 Detailed Description

Board GPIO macros and pinout definitions for the GPIO Actuator module.

Date

June 20, 2022

8.4.2 Macro Definition Documentation

8.4.2.1 GPIO_ACT_PIN_AUX

```
#define GPIO_ACT_PIN_AUX GPIO_NUM_8
```

GPIO pin number for auxiliary system relay.

System Auxiliary Relay

8.4.2.2 GPIO_ACT_PIN_CCW

```
#define GPIO_ACT_PIN_CCW GPIO_NUM_10
```

GPIO pin number for counter-clockwise direction.

Direction Counter-Clockwise

8.4.2.3 GPIO_ACT_PIN_CCW_IN

```
#define GPIO_ACT_PIN_CCW_IN GPIO_NUM_6
```

GPIO pin number for counter-clockwise input.

Counter-Clockwise Input

8.4.2.4 GPIO_ACT_PIN_CW

```
#define GPIO_ACT_PIN_CW GPIO_NUM_11
```

GPIO pin number for clockwise direction.

Direction Clockwise

8.4.2.5 GPIO_ACT_PIN_CW_IN

```
#define GPIO_ACT_PIN_CW_IN GPIO_NUM_7
```

GPIO pin number for clockwise input.

Clockwise Input

8.4.2.6 GPIO_ACT_PIN_OFF

```
#define GPIO_ACT_PIN_OFF GPIO_NUM_12
```

GPIO pin number for turning the system off.

Main system relay on

8.4.2.7 GPIO_ACT_PIN_ON

```
#define GPIO_ACT_PIN_ON GPIO_NUM_13
```

GPIO pin number for turning the system on.

Main system relay off

8.4.2.8 GPIO_ACT_PIN_PERC_AUX

```
#define GPIO_ACT_PIN_PERC_AUX GPIO_NUM_16
```

GPIO pin number for auxiliary relay for percentimeter.

Auxiliary Relay for Percentimeter

8.4.2.9 GPIO_ACT_PIN_PERC_IN

```
#define GPIO_ACT_PIN_PERC_IN GPIO_NUM_5
```

GPIO pin number for percentimeter read input.

Percentimeter Read Input

8.4.2.10 GPIO_ACT_PIN_PERC_OUT

```
#define GPIO_ACT_PIN_PERC_OUT GPIO_NUM_35
```

GPIO pin number for percentimeter reading output.

Percentimeter

8.4.2.11 GPIO_ACT_PIN_PRESS

```
#define GPIO_ACT_PIN_PRESS GPIO_NUM_4
```

GPIO pin number for water pressure reading input.

Water pressure reading

8.4.2.12 GPIO_ACT_PIN_PUMP

```
#define GPIO_ACT_PIN_PUMP GPIO_NUM_14
```

GPIO pin number for pump relay.

Pump Relay

8.4.2.13 GPIO_ACT_PIN_WATERING

```
#define GPIO_ACT_PIN_WATERING GPIO_NUM_9
```

GPIO pin number for watering relay.

Watering Relay

8.4.2.14 GPIO_INPUT_PIN_GROUP

```
#define GPIO_INPUT_PIN_GROUP
```

Value:

```
( (1ULL<<GPIO_ACT_PIN_CW_IN)      \
  | (1ULL<<GPIO_ACT_PIN_CCW_IN)    \
  | (1ULL<<GPIO_ACT_PIN_PRESS)) \
```

GPIO pin bitmask for all input pins.

8.4.2.15 GPIO_OUTPUT_PIN_GROUP

```
#define GPIO_OUTPUT_PIN_GROUP
```

Value:

```
( (1ULL<<GPIO_ACT_PIN_ON)          \
  | (1ULL<<GPIO_ACT_PIN_OFF)        \
  | (1ULL<<GPIO_ACT_PIN_AUX)        \
  | (1ULL<<GPIO_ACT_PIN_CW)         \
  | (1ULL<<GPIO_ACT_PIN_CCW)        \
  | (1ULL<<GPIO_ACT_PIN_WATERING)   \
  | (1ULL<<GPIO_ACT_PIN_PERC_AUX)   \
  | (1ULL<<GPIO_ACT_PIN_PERC_OUT)   \
  | (1ULL<<GPIO_ACT_PIN_PUMP)) \
```

GPIO pin bitmask for all output pins.

8.4.3 Function Documentation

8.4.3.1 gpio_actuator_get()

```
pivot_actions gpio_actuator_get (
    void )
```

Gets the current pivot configuration.

Returns

The current pivot configuration

8.4.3.2 gpio_actuator_init()

```
esp_err_t gpio_actuator_init (  
    void )
```

Initializes the GPIO actuator module.

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to initialize

Here is the caller graph for this function:



8.4.3.3 gpio_actuator_set()

```
esp_err_t gpio_actuator_set (  
    pivot_actions config )
```

Sets the pivot configuration.

Parameters

<i>config</i>	The pivot configuration to set
---------------	--------------------------------

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to set the configuration

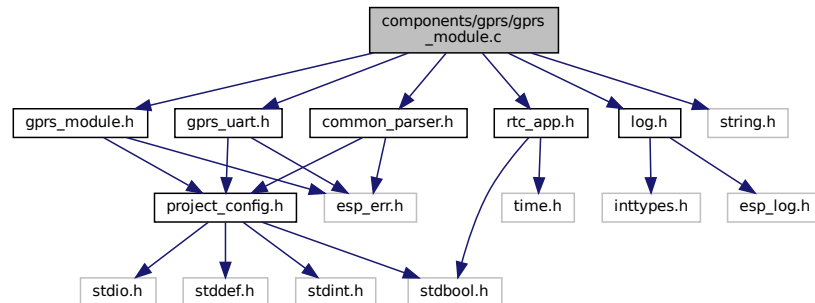
8.5 components/gprs/gprs_module.c File Reference

Controls the flow of reception and transmission of the GPRS module.

```
#include "gprs_module.h"  
#include "gprs_uart.h"  
#include "common_parser.h"
```

```
#include "rtc_app.h"
#include "log.h"
#include <string.h>
```

Include dependency graph for gprs_module.c:



Macros

- `#define GPRS_MODULE_TAG "gprs_module"`
Tag used for logging related to the GPRS module.

Functions

- `void gprs_module_call (const char *buffer, size_t buffer_size)`
Handles the GPRS module call.
- `esp_err_t gprs_module_init (void)`
Initializes the GPRS module.
- `esp_err_t gprs_module_send_event (pivot_actions config_in, uint16_t degree, const char *gprs_id)`
Sends an event through the GPRS module.
- `esp_err_t gprs_module_set_id (const char *gprs_id)`
Sets the GPRS ID.
- `esp_err_t gprs_module_send_idp (const char *gprs_id)`
Sends the GPRS IDP.
- `__attribute__((weak))`

8.5.1 Detailed Description

Controls the flow of reception and transmission of the GPRS module.

Date

June 21, 2022

8.5.2 Function Documentation

8.5.2.1 gprs_module_call()

```
void gprs_module_call (
    const char * buffer,
    size_t buffer_size )
```

Handles the GPRS module call.

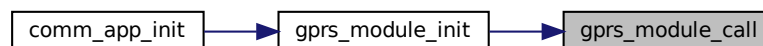
Function is triggered when something arrives in the UART.

This function is responsible for processing the call received from the GPRS module.

Parameters

<i>buffer</i>	The buffer containing the call data.
<i>buffer_size</i>	The size of the buffer.
<i>buffer[in]</i>	: received content
<i>buffer_size[in]</i>	: received content size

Here is the caller graph for this function:



8.5.2.2 gprs_module_init()

```
esp_err_t gprs_module_init (
    void )
```

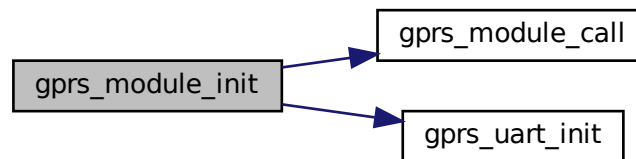
Initializes the GPRS module.

This function initializes the GPRS module and prepares it for communication.

Returns

- ESP_OK: Initialization successful.
- ESP_FAIL: Initialization failed.

Here is the call graph for this function:



Here is the caller graph for this function:



8.5.2.3 gprs_module_send_event()

```

esp_err_t gprs_module_send_event (
    pivot_actions config_in,
    uint16_t degree,
    const char * gprs_id )
  
```

Sends an event through the GPRS module.

This function converts the provided configuration structure to a string and sends it through the UART.

Parameters

<i>config_in</i>	Configuration structure to send.
<i>degree</i>	Degree value.
<i>gprs_id</i>	GPRS ID.

Returns

- ESP_OK: Event sent successfully.
- ESP_FAIL: Failed to send event.

8.5.2.4 gprs_module_send_idp()

```
esp_err_t gprs_module_send_idp (  
    const char * gprs_id )
```

Sends the GPRS IDP.

This function sends the GPRS IDP.

Parameters

<i>gprs_id</i>	GPRS ID to send.
----------------	------------------

Returns

- ESP_OK: GPRS IDP sent successfully.
- ESP_FAIL: Failed to send GPRS IDP.

Here is the call graph for this function:



8.5.2.5 gprs_module_set_id()

```
esp_err_t gprs_module_set_id (  
    const char * gprs_id )
```

Sets the GPRS ID.

This function sets the GPRS ID used for communication.

Parameters

<i>gprs_id</i>	GPRS ID to set.
----------------	-----------------

Returns

- ESP_OK: GPRS ID set successfully.
- ESP_FAIL: Failed to set GPRS ID.

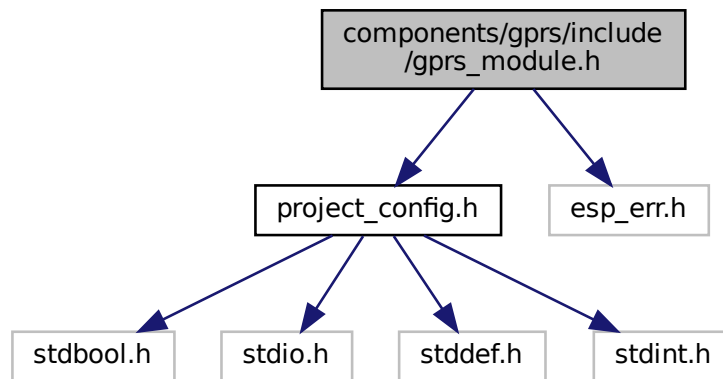
8.6 components/gprs/include/gprs_module.h File Reference

GPRS Module API.

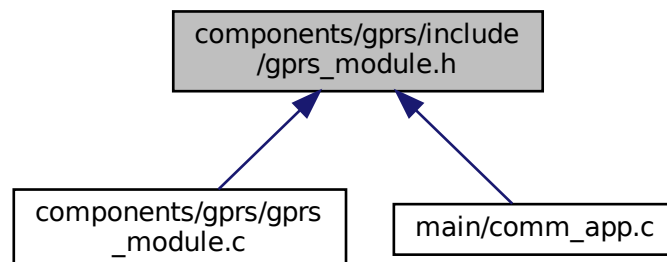
```
#include "project_config.h"
```

```
#include "esp_err.h"
```

Include dependency graph for gprs_module.h:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t gprs_module_init` (void)
Initializes the GPRS module.
- `esp_err_t gprs_module_send_event` (pivot_actions config_in, uint16_t degree, const char *gprs_id)
Sends an event through the GPRS module.
- `esp_err_t gprs_module_set_id` (const char *gprs_id)

- Sets the GPRS ID.*
 - `esp_err_t gprs_module_send_idp` (`const char *gprs_id`)
 - Sends the GPRS IDP.*
 - `time_t gprs_module_get_timestamp` (`void`)
 - Gets the timestamp in seconds.*
 - `void GPRS_MODULE_NOTIFY_APP` (`void *notify_buffer`)
 - Notifies the application about status messages.*

8.6.1 Detailed Description

GPRS Module API.

This file contains the declarations of functions and structures for the GPRS module.

8.6.2 Function Documentation

8.6.2.1 gprs_module_get_timestamp()

```
time_t gprs_module_get_timestamp (  
    void )
```

Gets the timestamp in seconds.

This function retrieves the timestamp obtained via GPRS.

Returns

Timestamp in seconds.

8.6.2.2 gprs_module_init()

```
esp_err_t gprs_module_init (  
    void )
```

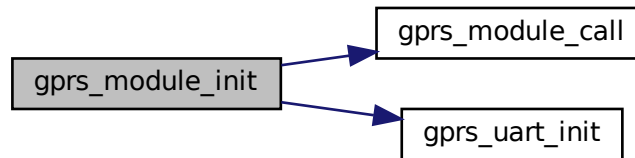
Initializes the GPRS module.

This function initializes the GPRS module and prepares it for communication.

Returns

- ESP_OK: Initialization successful.
- ESP_FAIL: Initialization failed.

Here is the call graph for this function:



Here is the caller graph for this function:



8.6.2.3 GPRS_MODULE_NOTIFY_APP()

```
void GPRS_MODULE_NOTIFY_APP (  
    void * notify_buffer )
```

Notifies the application about status messages.

This function triggers a callback for each status message received (write and read).

Parameters

<i>notify_buffer</i>	Pointer to the notification buffer.
----------------------	-------------------------------------

8.6.2.4 gprs_module_send_event()

```
esp_err_t gprs_module_send_event (
```

```
    pivot_actions config_in,  
    uint16_t degree,  
    const char * gprs_id )
```

Sends an event through the GPRS module.

This function converts the provided configuration structure to a string and sends it through the UART.

Parameters

<i>config_in</i>	Configuration structure to send.
<i>degree</i>	Degree value.
<i>gprs_id</i>	GPRS ID.

Returns

- ESP_OK: Event sent successfully.
- ESP_FAIL: Failed to send event.

8.6.2.5 gprs_module_send_idp()

```
esp_err_t gprs_module_send_idp (  
    const char * gprs_id )
```

Sends the GPRS IDP.

This function sends the GPRS IDP.

Parameters

<i>gprs_id</i>	GPRS ID to send.
----------------	------------------

Returns

- ESP_OK: GPRS IDP sent successfully.
- ESP_FAIL: Failed to send GPRS IDP.

Here is the call graph for this function:



8.6.2.6 gprs_module_set_id()

```
esp_err_t gprs_module_set_id (  
    const char * gprs_id )
```

Sets the GPRS ID.

This function sets the GPRS ID used for communication.

Parameters

<i>gprs_id</i>	GPRS ID to set.
----------------	-----------------

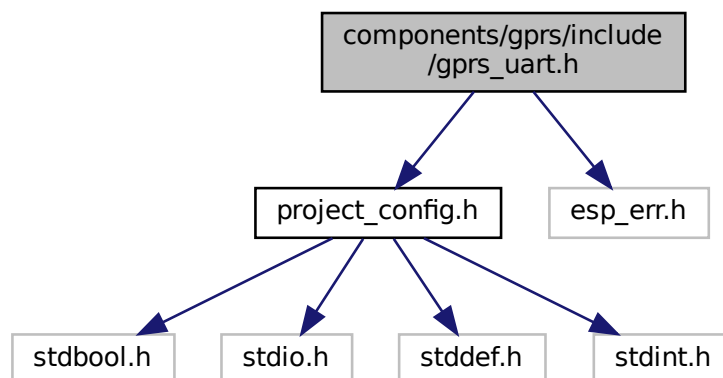
Returns

- ESP_OK: GPRS ID set successfully.
- ESP_FAIL: Failed to set GPRS ID.

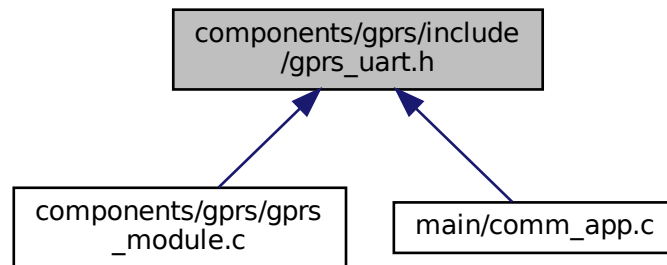
8.7 components/gprs/include/gprs_uart.h File Reference

Header file for GPRS UART communication functions.

```
#include "project_config.h"  
#include "esp_err.h"  
Include dependency graph for gprs_uart.h:
```



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef void(* [gprs_uart_callback](#)) (const char *buffer, size_t buffer_size)
Callback function used for receiving data from the GPRS UART module.

Functions

- esp_err_t [gprs_uart_init](#) (const [gprs_uart_callback](#) callback)
Initialize the GPRS UART module.
- esp_err_t [gprs_uart_send_event](#) (const char *event, size_t event_size)
Send an event through the GPRS UART module.

8.7.1 Detailed Description

Header file for GPRS UART communication functions.

Date

7 de agosto de 2022

8.7.2 Typedef Documentation

8.7.2.1 gprs_uart_callback

```
typedef void(* gprs_uart_callback) (const char *buffer, size_t buffer_size)
```

Callback function used for receiving data from the GPRS UART module.

Parameters

<i>buffer</i>	The received data buffer
<i>buffer_size</i>	The size of the received data buffer

8.7.3 Function Documentation

8.7.3.1 gprs_uart_init()

```
esp_err_t gprs_uart_init (  
    const gprs_uart_callback callback )
```

Initialize the GPRS UART module.

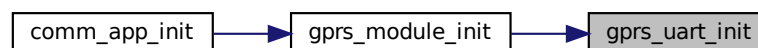
Parameters

<i>callback</i>	The callback function to be called when data is received
-----------------	--

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to initialize
- ESP_ERR_INVALID_ARG: Invalid callback function

Here is the caller graph for this function:



8.7.3.2 gprs_uart_send_event()

```
esp_err_t gprs_uart_send_event (  
    const char * event,  
    size_t event_size )
```

Send an event through the GPRS UART module.

Parameters

<i>event</i>	The event buffer to be sent
<i>event_size</i>	The size of the event buffer

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to send the event

Here is the caller graph for this function:

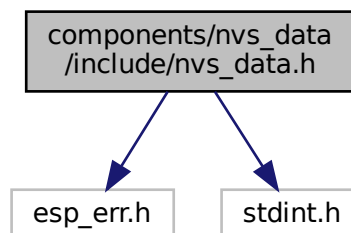


8.8 components/nvs_data/include/nvs_data.h File Reference

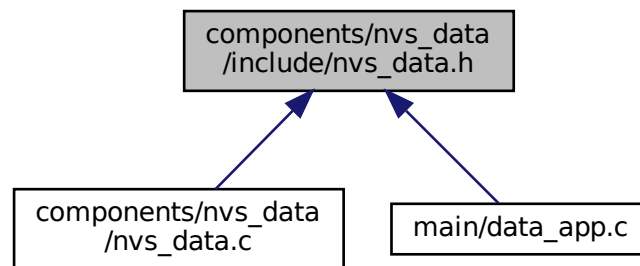
Class for direct access to flash memory (NVS).

```
#include "esp_err.h"
#include <stdint.h>
```

Include dependency graph for `nvs_data.h`:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t nvs_data_init` (void)
Initializes the NVS data class.
- `esp_err_t nvs_data_set` (const char *label_name, const char *key, const void *value, size_t length)
Sets the value in the NVS data.
- `size_t nvs_data_get_size` (const char *label_name, const char *key)
Gets the size of the NVS data.
- `esp_err_t nvs_data_get_blob` (const char *label_name, const char *key, uint8_t *out_value)
Gets the NVS data as a blob.

8.8.1 Detailed Description

Class for direct access to flash memory (NVS).

class for direct access to flash memory (NVS)

Date

Set 18, 2022

8.8.2 Function Documentation

8.8.2.1 nvs_data_get_blob()

```

esp_err_t nvs_data_get_blob (
    const char * label_name,
    const char * key,
    uint8_t * out_value )
  
```

Gets the NVS data as a blob.

This function retrieves the NVS data as a blob under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key for which to retrieve the data.
<i>out_value</i>	The buffer to store the retrieved data.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.8.2.2 nvs_data_get_size()

```
size_t nvs_data_get_size (
    const char * label_name,
    const char * key )
```

Gets the size of the NVS data.

This function retrieves the size of the NVS data under the specified label and key.

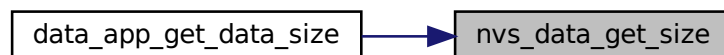
Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key for which to retrieve the size.

Returns

`size_t` The size of the NVS data.

Here is the caller graph for this function:

**8.8.2.3 nvs_data_init()**

```
esp_err_t nvs_data_init (
    void )
```

Initializes the NVS data class.

This function initializes the NVS data class by initializing the NVS flash memory.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

8.8.2.4 nvs_data_set()

```
esp_err_t nvs_data_set (
    const char * label_name,
    const char * key,
    const void * value,
    size_t length )
```

Sets the value in the NVS data.

This function sets the value in the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key under which the value is stored.
<i>value</i>	The value to be stored.
<i>length</i>	The length of the value.

Returns

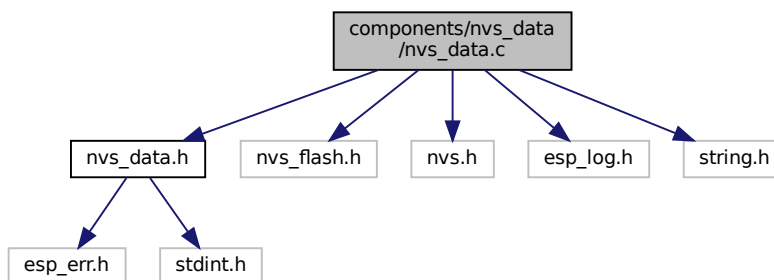
`esp_err_t` An error code indicating the success or failure of the operation.

8.9 components/nvs_data/nvs_data.c File Reference

Class for direct access to flash memory (NVS).

```
#include "nvs_data.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"
#include "string.h"
```

Include dependency graph for `nvs_data.c`:



Macros

- `#define NVS_DATA_TAG "nvs_data"`
Tag used for logging related to NVS data.

Functions

- `esp_err_t nvs_data_init (void)`
Initializes the NVS data class.
- `esp_err_t nvs_data_set (const char *label_name, const char *key, const void *value, size_t length)`
Sets the value in the NVS data.
- `size_t nvs_data_get_size (const char *label_name, const char *key)`
Gets the size of the NVS data.
- `esp_err_t nvs_data_get_blob (const char *label_name, const char *key, uint8_t *out_value)`
Gets the NVS data as a blob.

8.9.1 Detailed Description

Class for direct access to flash memory (NVS).

8.9.2 Function Documentation

8.9.2.1 nvs_data_get_blob()

```
esp_err_t nvs_data_get_blob (  
    const char * label_name,  
    const char * key,  
    uint8_t * out_value )
```

Gets the NVS data as a blob.

This function retrieves the NVS data as a blob under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key for which to retrieve the data.
<i>out_value</i>	The buffer to store the retrieved data.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.9.2.2 nvs_data_get_size()

```
size_t nvs_data_get_size (
    const char * label_name,
    const char * key )
```

Gets the size of the NVS data.

This function retrieves the size of the NVS data under the specified label and key.

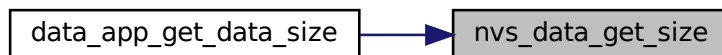
Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key for which to retrieve the size.

Returns

size_t The size of the NVS data.

Here is the caller graph for this function:



8.9.2.3 nvs_data_init()

```
esp_err_t nvs_data_init (
    void )
```

Initializes the NVS data class.

This function initializes the NVS data class by initializing the NVS flash memory.

Returns

esp_err_t An error code indicating the success or failure of the initialization.

8.9.2.4 nvs_data_set()

```
esp_err_t nvs_data_set (
    const char * label_name,
    const char * key,
    const void * value,
    size_t length )
```

Sets the value in the NVS data.

This function sets the value in the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>key</i>	The key under which the value is stored.
<i>value</i>	The value to be stored.
<i>length</i>	The length of the value.

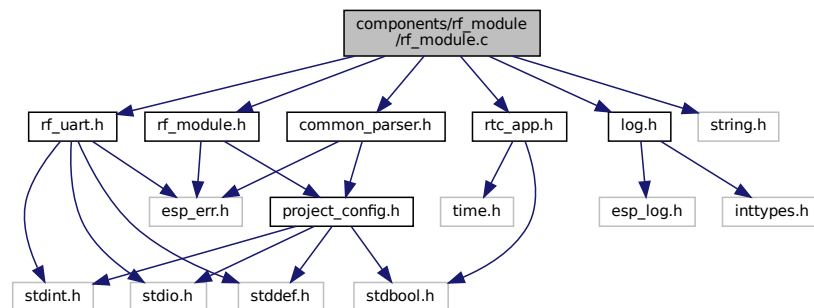
Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.10 components/rf_module/rf_module.c File Reference

controls the flow of reception and transmission of the RF module

```
#include "rf_module.h"
#include "rf_uart.h"
#include "common_parser.h"
#include "rtc_app.h"
#include "log.h"
#include <string.h>
Include dependency graph for rf_module.c:
```



Macros

- `#define RF_MODULE_TAG "rf_module"`

Functions

- `void rf_module_call (char *buffer, size_t buffer_size)`
Function is triggered when something arrives in the UART.
- `esp_err_t rf_module_init (void)`
start the RF module
- `esp_err_t rf_module_send_event (pivot_actions config_in)`
convert the structure to string and send it through the UART
- `uint16_t rf_module_get_angle (void)`
get the angle in degrees
- `__attribute__((weak))`

8.10.1 Detailed Description

controls the flow of reception and transmission of the RF module

Date

June 21, 2022

8.10.2 Function Documentation

8.10.2.1 rf_module_call()

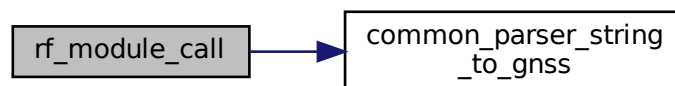
```
void rf_module_call (
    char * buffer,
    size_t buffer_size )
```

Function is triggered when something arrives in the UART.

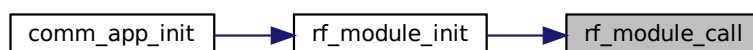
Parameters

<i>buffer[in]</i>	: received content
<i>buffer_size[in]</i>	: received content size

Here is the call graph for this function:



Here is the caller graph for this function:



8.10.2.2 rf_module_get_angle()

```
uint16_t rf_module_get_angle (  
    void )
```

get the angle in degrees

Returns

angle: returns the current angle (degrees) received

Here is the caller graph for this function:



8.10.2.3 rf_module_init()

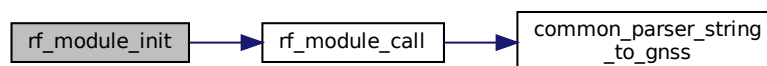
```
esp_err_t rf_module_init (  
    void )
```

start the RF module

Returns

- ESP_OK: success
- ESP_FAIL: fail

Here is the call graph for this function:



Here is the caller graph for this function:



8.10.2.4 rf_module_send_event()

```
esp_err_t rf_module_send_event (
    pivot_actions config_in )
```

convert the structure to string and send it through the UART

Parameters

<code>config_in[in]</code>	configuration structure to send
----------------------------	---------------------------------

Returns

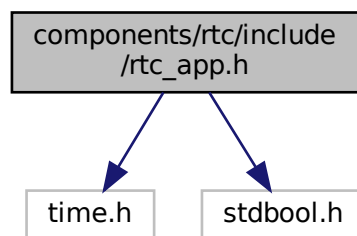
- ESP_OK: success
- ESP_FAIL: fail

8.11 components/rtc/include/rtc_app.h File Reference

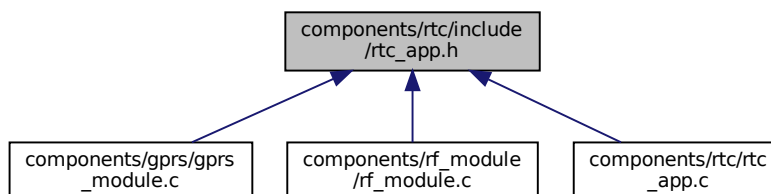
RTC application class for managing date and time functionality.

```
#include <time.h>
#include <stdbool.h>
```

Include dependency graph for rtc_app.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define [RTC_CONFIG_TIMEZONE](#) -3
The default time zone for the RTC configuration.

Functions

- bool [rtc_app_init](#) (void)
Initializes the RTC application.
- bool [rtc_app_set_timestamp](#) (time_t timestamp)
Sets the timestamp in the RTC.
- time_t [rtc_app_get_timestamp](#) (bool rtc_show_dt)
Gets the timestamp from the RTC.
- void [rtc_app_get_date_time](#) (struct tm *rtcinfo)
Gets the date and time from the RTC.
- void [rtc_show_date_time](#) (time_t timestamp_now, uint8_t time_z)
Displays the date and time.

8.11.1 Detailed Description

RTC application class for managing date and time functionality.

This header file defines the RTC application class for managing date and time functionality.

8.11.2 Function Documentation

8.11.2.1 [rtc_app_get_date_time\(\)](#)

```
void rtc_app_get_date_time (
    struct tm * rtcinfo )
```

Gets the date and time from the RTC.

This function retrieves the date and time from the RTC and stores it in the specified time structure.

Parameters

rtcinfo	The pointer to the time structure to store the date and time.
-------------------------	---

8.11.2.2 [rtc_app_get_timestamp\(\)](#)

```
time_t rtc_app_get_timestamp (
    bool rtc_show_dt )
```

Gets the timestamp from the RTC.

This function retrieves the timestamp from the RTC.

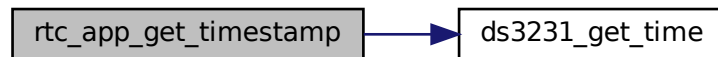
Parameters

<code>rtc_show↔ _dt</code>	Flag indicating whether to display the date and time in local time.
--------------------------------	---

Returns

`time_t` The timestamp value.

Here is the call graph for this function:



8.11.2.3 rtc_app_init()

```
bool rtc_app_init (  
    void )
```

Initializes the RTC application.

This function initializes the RTC application.

Returns

`bool` True if initialization is successful, false otherwise.

Here is the call graph for this function:



8.11.2.4 rtc_app_set_timestamp()

```
bool rtc_app_set_timestamp (
    time_t timestamp )
```

Sets the timestamp in the RTC.

This function sets the timestamp in the RTC with the specified timestamp value.

Parameters

<i>timestamp</i>	The timestamp value to set.
------------------	-----------------------------

Returns

bool True if setting the timestamp is successful, false otherwise.

8.11.2.5 rtc_show_date_time()

```
void rtc_show_date_time (
    time_t timestamp_now,
    uint8_t time_z )
```

Displays the date and time.

This function displays the date and time based on the specified timestamp and time zone.

Parameters

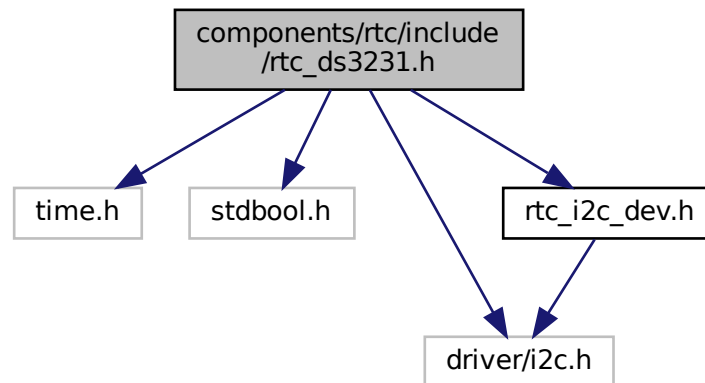
<i>timestamp_now</i>	The timestamp value.
<i>time_z</i>	The time zone.

8.12 components/rtc/include/rtc_ds3231.h File Reference

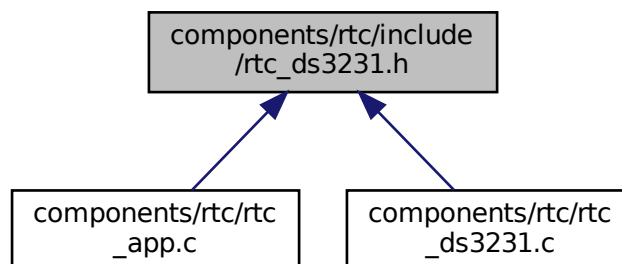
RTC DS3231 class for interfacing with the DS3231 RTC module.

```
#include <time.h>
#include <stdbool.h>
#include "driver/i2c.h"
#include "rtc_i2c_dev.h"
```

Include dependency graph for rtc_ds3231.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define DS3231_ADDR 0x68`
The I2C address of the DS3231 RTC module.
- `#define DS3231_STAT_OSCILLATOR 0x80`
Status bit indicating the oscillator stop flag in the DS3231.
- `#define DS3231_STAT_32KHZ 0x08`
Status bit indicating the 32kHz output flag in the DS3231.
- `#define DS3231_STAT_BUSY 0x04`
Status bit indicating the busy flag in the DS3231.
- `#define DS3231_STAT_ALARM_2 0x02`
Status bit indicating the alarm 2 flag in the DS3231.
- `#define DS3231_STAT_ALARM_1 0x01`

- Status bit indicating the alarm 1 flag in the DS3231.*

 - #define [DS3231_CTRL_OSCILLATOR](#) 0x80
- Control bit for enabling/disabling the oscillator in the DS3231.*

 - #define [DS3231_CTRL_SQUAREWAVE_BB](#) 0x40
- Control bit for enabling/disabling the square wave output in the DS3231.*

 - #define [DS3231_CTRL_TEMPCONV](#) 0x20
- Control bit for initiating temperature conversion in the DS3231.*

 - #define [DS3231_CTRL_ALARM_INTS](#) 0x04
- Control bit for enabling/disabling alarm interrupts in the DS3231.*

 - #define [DS3231_CTRL_ALARM2_INT](#) 0x02
- Control bit for enabling/disabling alarm 2 interrupt in the DS3231.*

 - #define [DS3231_CTRL_ALARM1_INT](#) 0x01
- Control bit for enabling/disabling alarm 1 interrupt in the DS3231.*

 - #define [DS3231_ALARM_WDAY](#) 0x40
- Flag indicating that the alarm is set to match both the date and weekday in the DS3231.*

 - #define [DS3231_ALARM_NOTSET](#) 0x80
- Flag indicating that the alarm is not set in the DS3231.*

 - #define [DS3231_ADDR_TIME](#) 0x00
- The address of the time registers in the DS3231.*

 - #define [DS3231_ADDR_ALARM1](#) 0x07
- The address of the alarm 1 registers in the DS3231.*

 - #define [DS3231_ADDR_ALARM2](#) 0x0b
- The address of the alarm 2 registers in the DS3231.*

 - #define [DS3231_ADDR_CONTROL](#) 0x0e
- The address of the control register in the DS3231.*

 - #define [DS3231_ADDR_STATUS](#) 0x0f
- The address of the status register in the DS3231.*

 - #define [DS3231_ADDR_AGING](#) 0x10
- The address of the aging offset register in the DS3231.*

 - #define [DS3231_ADDR_TEMP](#) 0x11
- The address of the temperature registers in the DS3231.*

 - #define [DS3231_12HOUR_FLAG](#) 0x40
- Flag indicating 12-hour mode in the DS3231.*

 - #define [DS3231_12HOUR_MASK](#) 0x1f
- Mask for extracting the hour value in 12-hour mode from the DS3231.*

 - #define [DS3231_PM_FLAG](#) 0x20
- Flag indicating PM (post meridiem) in 12-hour mode in the DS3231.*

 - #define [DS3231_MONTH_MASK](#) 0x1f
- Mask for extracting the month value from the DS3231.*

Functions

- [uint8_t bcd2dec](#) (uint8_t val)

Converts a binary-coded decimal (BCD) value to decimal.
- [uint8_t dec2bcd](#) (uint8_t val)

Converts a decimal value to binary-coded decimal (BCD).
- [esp_err_t ds3231_init_desc](#) (i2c_dev_t *dev, i2c_port_t port, gpio_num_t sda_gpio, gpio_num_t scl_gpio)

Initializes the DS3231 RTC descriptor.
- [esp_err_t ds3231_set_time](#) (i2c_dev_t *dev, struct tm *time)

Sets the time on the DS3231 RTC module.

- `esp_err_t ds3231_get_raw_temp (i2c_dev_t *dev, int16_t *temp)`
Gets the raw temperature value from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_integer (i2c_dev_t *dev, int8_t *temp)`
Gets the integer part of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_float (i2c_dev_t *dev, float *temp)`
Gets the floating-point representation of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_time (i2c_dev_t *dev, struct tm *time)`
Gets the current time from the DS3231 RTC module.

8.12.1 Detailed Description

RTC DS3231 class for interfacing with the DS3231 RTC module.

This header file defines the RTC DS3231 class and its associated functions for interfacing with the DS3231 RTC module.

8.12.2 Function Documentation

8.12.2.1 bcd2dec()

```
uint8_t bcd2dec (
    uint8_t val )
```

Converts a binary-coded decimal (BCD) value to decimal.

Parameters

<i>val</i>	The BCD value to convert.
------------	---------------------------

Returns

`uint8_t` The decimal value.

8.12.2.2 dec2bcd()

```
uint8_t dec2bcd (
    uint8_t val )
```

Converts a decimal value to binary-coded decimal (BCD).

Parameters

<i>val</i>	The decimal value to convert.
------------	-------------------------------

Returns

uint8_t The BCD value.

8.12.2.3 ds3231_get_raw_temp()

```
esp_err_t ds3231_get_raw_temp (
    i2c_dev_t * dev,
    int16_t * temp )
```

Gets the raw temperature value from the DS3231 RTC module.

This function retrieves the raw temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

esp_err_t An error code indicating the success or failure of the operation.

8.12.2.4 ds3231_get_temp_float()

```
esp_err_t ds3231_get_temp_float (
    i2c_dev_t * dev,
    float * temp )
```

Gets the floating-point representation of the temperature from the DS3231 RTC module.

This function retrieves the floating-point representation of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

esp_err_t An error code indicating the success or failure of the operation.

8.12.2.5 ds3231_get_temp_integer()

```
esp_err_t ds3231_get_temp_integer (
    i2c_dev_t * dev,
    int8_t * temp )
```

Gets the integer part of the temperature from the DS3231 RTC module.

This function retrieves the integer part of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.12.2.6 ds3231_get_time()

```
esp_err_t ds3231_get_time (
    i2c_dev_t * dev,
    struct tm * time )
```

Gets the current time from the DS3231 RTC module.

This function retrieves the current time from the DS3231 RTC module and stores it in the specified time structure.

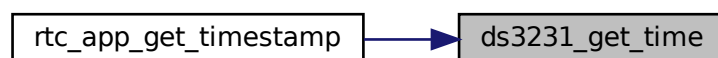
Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure to store the current time.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Here is the caller graph for this function:



8.12.2.7 ds3231_init_desc()

```
esp_err_t ds3231_init_desc (
    i2c_dev_t * dev,
    i2c_port_t port,
    gpio_num_t sda_gpio,
    gpio_num_t scl_gpio )
```

Initializes the DS3231 RTC descriptor.

This function initializes the DS3231 RTC descriptor with the specified I2C port and GPIO pin numbers for SDA and SCL.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>port</i>	The I2C port number.
<i>sda_gpio</i>	The GPIO number for the I2C SDA signal.
<i>scl_gpio</i>	The GPIO number for the I2C SCL signal.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

Here is the caller graph for this function:



8.12.2.8 ds3231_set_time()

```
esp_err_t ds3231_set_time (
    i2c_dev_t * dev,
    struct tm * time )
```

Sets the time on the DS3231 RTC module.

This function sets the time on the DS3231 RTC module using the specified time structure.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure containing the new time.

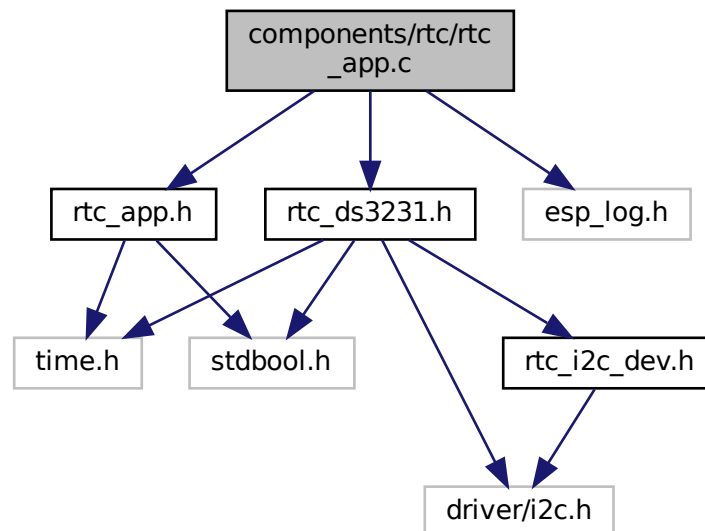
Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.13 components/rtc/rtc_app.c File Reference

RTC application implementation for managing date and time functionality.

```
#include "rtc_app.h"
#include "rtc_ds3231.h"
#include "esp_log.h"
Include dependency graph for rtc_app.c:
```



Macros

- `#define RTC_APP_TAG "rtc_app"`
- `#define RTC_SDA_PIN 36`
The GPIO pin number for the RTC I2C SDA signal.
- `#define RTC_SCL_PIN 37`
The GPIO pin number for the RTC I2C SCL signal.

Functions

- bool [rtc_app_init](#) (void)
Initializes the RTC application.
- bool [rtc_app_set_timestamp](#) (time_t timestamp)
Sets the timestamp in the RTC.
- time_t [rtc_app_get_timestamp](#) (bool rtc_show_dt)
Gets the timestamp from the RTC.
- void [rtc_app_get_date_time](#) (struct tm *rtcinfo)
Gets the date and time from the RTC.
- void [rtc_show_date_time](#) (time_t timestamp_now, uint8_t time_z)
Displays the date and time.

8.13.1 Detailed Description

RTC application implementation for managing date and time functionality.

This file contains the implementation of the RTC application for managing date and time functionality.

8.13.2 Function Documentation

8.13.2.1 [rtc_app_get_date_time\(\)](#)

```
void rtc_app_get_date_time (
    struct tm * rtcinfo )
```

Gets the date and time from the RTC.

This function retrieves the date and time from the RTC and stores it in the specified time structure.

Parameters

rtcinfo	The pointer to the time structure to store the date and time.
-------------------------	---

8.13.2.2 [rtc_app_get_timestamp\(\)](#)

```
time_t rtc_app_get_timestamp (
    bool rtc_show_dt )
```

Gets the timestamp from the RTC.

This function retrieves the timestamp from the RTC.

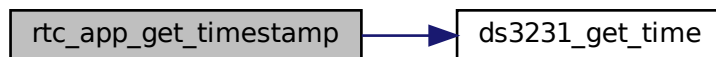
Parameters

<code>rtc_show_dt</code>	Flag indicating whether to display the date and time in local time.
--------------------------	---

Returns

`time_t` The timestamp value.

Here is the call graph for this function:

**8.13.2.3 rtc_app_init()**

```
bool rtc_app_init (
    void )
```

Initializes the RTC application.

This function initializes the RTC application.

Returns

`bool` True if initialization is successful, false otherwise.

Here is the call graph for this function:

**8.13.2.4 rtc_app_set_timestamp()**

```
bool rtc_app_set_timestamp (
    time_t timestamp )
```

Sets the timestamp in the RTC.

This function sets the timestamp in the RTC with the specified timestamp value.

Parameters

<i>timestamp</i>	The timestamp value to set.
------------------	-----------------------------

Returns

bool True if setting the timestamp is successful, false otherwise.

8.13.2.5 rtc_show_date_time()

```
void rtc_show_date_time (
    time_t timestamp_now,
    uint8_t time_z )
```

Displays the date and time.

This function displays the date and time based on the specified timestamp and time zone.

Parameters

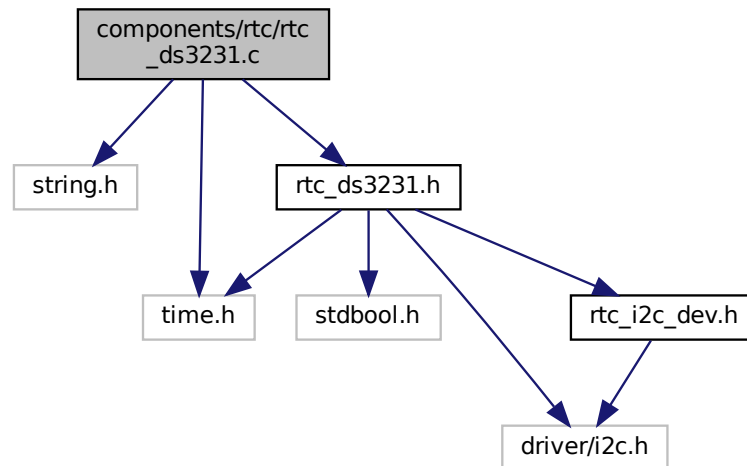
<i>timestamp_now</i>	The timestamp value.
<i>time_z</i>	The time zone.

8.14 components/rtc/rtc_ds3231.c File Reference

RTC DS3231 class implementation.

```
#include <string.h>
#include <time.h>
#include "rtc_ds3231.h"
```

Include dependency graph for rtc_ds3231.c:



Macros

- `#define CHECK_ARG(ARG) do { if (!ARG) return ESP_ERR_INVALID_ARG; } while (0)`
Checks if the argument is valid.

Functions

- `uint8_t bcd2dec (uint8_t val)`
Converts a binary-coded decimal (BCD) value to decimal.
- `uint8_t dec2bcd (uint8_t val)`
Converts a decimal value to binary-coded decimal (BCD).
- `esp_err_t ds3231_init_desc (i2c_dev_t *dev, i2c_port_t port, gpio_num_t sda_gpio, gpio_num_t scl_gpio)`
Initializes the DS3231 RTC descriptor.
- `esp_err_t ds3231_set_time (i2c_dev_t *dev, struct tm *time)`
Sets the time on the DS3231 RTC module.
- `esp_err_t ds3231_get_raw_temp (i2c_dev_t *dev, int16_t *temp)`
Gets the raw temperature value from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_integer (i2c_dev_t *dev, int8_t *temp)`
Gets the integer part of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_float (i2c_dev_t *dev, float *temp)`
Gets the floating-point representation of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_time (i2c_dev_t *dev, struct tm *time)`
Gets the current time from the DS3231 RTC module.

8.14.1 Detailed Description

RTC DS3231 class implementation.

This file contains the implementation of the RTC DS3231 class for interfacing with the DS3231 RTC module.

8.14.2 Macro Definition Documentation

8.14.2.1 CHECK_ARG

```
#define CHECK_ARG(  
    ARG ) do { if (!ARG) return ESP_ERR_INVALID_ARG; } while (0)
```

Checks if the argument is valid.

This macro checks if the argument is valid. If the argument is not valid, it returns ESP_ERR_INVALID_ARG.

Parameters

<i>ARG</i>	The argument to check.
------------	------------------------

8.14.3 Function Documentation

8.14.3.1 bcd2dec()

```
uint8_t bcd2dec (  
    uint8_t val )
```

Converts a binary-coded decimal (BCD) value to decimal.

Parameters

<i>val</i>	The BCD value to convert.
------------	---------------------------

Returns

uint8_t The decimal value.

8.14.3.2 dec2bcd()

```
uint8_t dec2bcd (  
    uint8_t val )
```

Converts a decimal value to binary-coded decimal (BCD).

Parameters

<i>val</i>	The decimal value to convert.
------------	-------------------------------

Returns

uint8_t The BCD value.

8.14.3.3 ds3231_get_raw_temp()

```
esp_err_t ds3231_get_raw_temp (
    i2c_dev_t * dev,
    int16_t * temp )
```

Gets the raw temperature value from the DS3231 RTC module.

This function retrieves the raw temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

esp_err_t An error code indicating the success or failure of the operation.

8.14.3.4 ds3231_get_temp_float()

```
esp_err_t ds3231_get_temp_float (
    i2c_dev_t * dev,
    float * temp )
```

Gets the floating-point representation of the temperature from the DS3231 RTC module.

This function retrieves the floating-point representation of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.14.3.5 ds3231_get_temp_integer()

```
esp_err_t ds3231_get_temp_integer (
    i2c_dev_t * dev,
    int8_t * temp )
```

Gets the integer part of the temperature from the DS3231 RTC module.

This function retrieves the integer part of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.14.3.6 ds3231_get_time()

```
esp_err_t ds3231_get_time (
    i2c_dev_t * dev,
    struct tm * time )
```

Gets the current time from the DS3231 RTC module.

This function retrieves the current time from the DS3231 RTC module and stores it in the specified time structure.

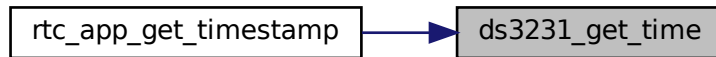
Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure to store the current time.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Here is the caller graph for this function:

**8.14.3.7 ds3231_init_desc()**

```

esp_err_t ds3231_init_desc (
    i2c_dev_t * dev,
    i2c_port_t port,
    gpio_num_t sda_gpio,
    gpio_num_t scl_gpio )
  
```

Initializes the DS3231 RTC descriptor.

This function initializes the DS3231 RTC descriptor with the specified I2C port and GPIO pin numbers for SDA and SCL.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>port</i>	The I2C port number.
<i>sda_gpio</i>	The GPIO number for the I2C SDA signal.
<i>scl_gpio</i>	The GPIO number for the I2C SCL signal.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

Here is the caller graph for this function:



8.14.3.8 ds3231_set_time()

```
esp_err_t ds3231_set_time (
    i2c_dev_t * dev,
    struct tm * time )
```

Sets the time on the DS3231 RTC module.

This function sets the time on the DS3231 RTC module using the specified time structure.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure containing the new time.

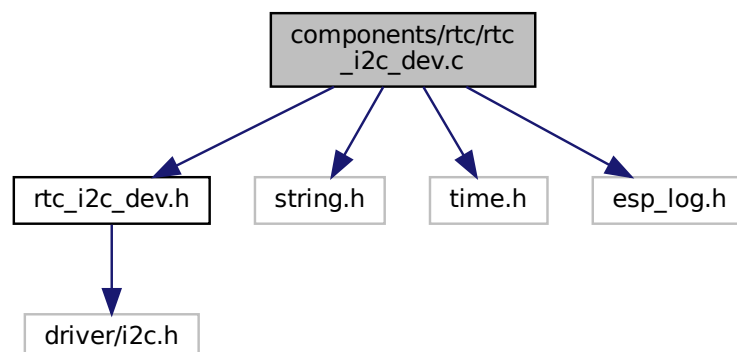
Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.15 components/rtc/rtc_i2c_dev.c File Reference

RTC I2C device implementation.

```
#include "rtc_i2c_dev.h"
#include <string.h>
#include <time.h>
#include "esp_log.h"
Include dependency graph for rtc_i2c_dev.c:
```



Macros

- `#define RTC_I2C_DEV_TAG "I2CDEV"`
The RTC_I2C_DEV_TAG used for logging related to the RTC I2C device.

Functions

- `esp_err_t i2c_master_init (i2c_port_t port, int sda, int scl)`
Initializes the I2C master for the specified port.
- `esp_err_t i2c_dev_read (const i2c_dev_t *dev, const void *out_data, size_t out_size, void *in_data, size_t in_size)`
Reads data from the I2C device.
- `esp_err_t i2c_dev_write (const i2c_dev_t *dev, const void *out_reg, size_t out_reg_size, const void *out_data, size_t out_size)`
Writes data to the I2C device.

8.15.1 Detailed Description

RTC I2C device implementation.

This file contains the implementation of the RTC I2C device.

8.15.2 Function Documentation

8.15.2.1 i2c_dev_read()

```
esp_err_t i2c_dev_read (
    const i2c_dev_t * dev,
    const void * out_data,
    size_t out_size,
    void * in_data,
    size_t in_size )
```

Reads data from the I2C device.

This function reads data from the I2C device specified by the given device structure.

Parameters

<i>dev</i>	The pointer to the I2C device structure.
<i>out_data</i>	The buffer containing the data to be written before the read operation.
<i>out_size</i>	The size of the output data buffer.
<i>in_data</i>	The buffer to store the read data.
<i>in_size</i>	The size of the input data buffer.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.15.2.2 i2c_dev_write()

```
esp_err_t i2c_dev_write (
    const i2c_dev_t * dev,
    const void * out_reg,
    size_t out_reg_size,
    const void * out_data,
    size_t out_size )
```

Writes data to the I2C device.

This function writes data to the I2C device specified by the given device structure.

Parameters

<i>dev</i>	The pointer to the I2C device structure.
<i>out_reg</i>	The buffer containing the register address before the write operation.
<i>out_reg_size</i>	The size of the register address buffer.
<i>out_data</i>	The buffer containing the data to be written.
<i>out_size</i>	The size of the output data buffer.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

8.15.2.3 i2c_master_init()

```
esp_err_t i2c_master_init (
    i2c_port_t port,
    int sda,
    int scl )
```

Initializes the I2C master for the specified port.

This function initializes the I2C master for the specified port with the given SDA and SCL pin numbers.

Parameters

<i>port</i>	The I2C port number.
<i>sda</i>	The GPIO number for the I2C SDA signal.
<i>scl</i>	The GPIO number for the I2C SCL signal.

Returns

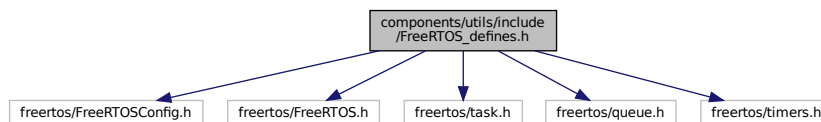
`esp_err_t` An error code indicating the success or failure of the initialization.

8.16 components/utils/include/FreeRTOS_defines.h File Reference

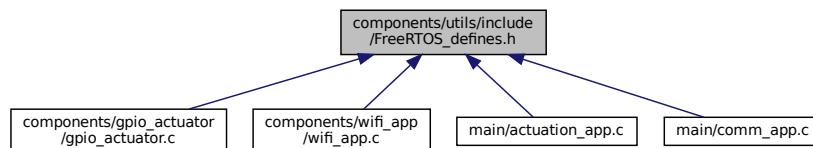
Defines for FreeRTOS tasks and priorities.

```
#include "freertos/FreeRTOSConfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/timers.h"
```

Include dependency graph for FreeRTOS_defines.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define MAIN_APP_TASK_1_NAME` "main sectorization task"
Name of the main sectorization task.
- `#define MAIN_APP_STACK_1_SIZE` (configMINIMAL_STACK_SIZE * 4)
Stack size of the main sectorization task.
- `#define MAIN_APP_TASK_1_PRIORITY` (tskIDLE_PRIORITY + 4)
Priority of the main sectorization task.
- `#define MAIN_APP_TASK_2_NAME` "main peak hours task"
Name of the main peak hours task.
- `#define MAIN_APP_STACK_2_SIZE` (configMINIMAL_STACK_SIZE * 4)
Stack size of the main peak hours task.
- `#define MAIN_APP_TASK_2_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the main peak hours task.
- `#define MAIN_APP_TASK_3_NAME` "main scheduling task"
Name of the main scheduling task.

- #define [MAIN_APP_STACK_3_SIZE](#) (configMINIMAL_STACK_SIZE * 6)
Stack size of the main scheduling task.
- #define [MAIN_APP_TASK_3_PRIORITY](#) (tskIDLE_PRIORITY + 3)
Priority of the main scheduling task.
- #define [DATA_APP_TASK_NAME](#) "data app task"
Name of the data app task.
- #define [DATA_APP_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 6)
Stack size of the data app task.
- #define [DATA_APP_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 2)
Priority of the data app task.
- #define [COMM_APP_TASK_NAME](#) "comm app task"
Name of the comm app task.
- #define [COMM_APP_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 10)
Stack size of the comm app task.
- #define [COMM_APP_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 2)
Priority of the comm app task.
- #define [ACTUATION_APP_TASK_NAME](#) "actuation app task"
Name of the actuation app task.
- #define [ACTUATION_APP_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuation app task.
- #define [ACTUATION_APP_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 3)
Priority of the actuation app task.
- #define [ACTUATOR_CHECK_TASK_NAME](#) "actuator check task"
Name of the actuator check task.
- #define [ACTUATOR_CHECK_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuator check task.
- #define [ACTUATOR_CHECK_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 3)
Priority of the actuator check task.
- #define [ACTUATOR_PERCENT_TASK_NAME](#) "actuator percent task"
Name of the actuator percent task.
- #define [ACTUATOR_PERCENT_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuator percent task.
- #define [ACTUATOR_PERCENT_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 3)
Priority of the actuator percent task.
- #define [GPRS_UART_TASK_NAME](#) "GPRS UART task"
Name of the GPRS UART task.
- #define [GPRS_UART_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 10)
Stack size of the GPRS UART task.
- #define [GPRS_UART_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 12)
Priority of the GPRS UART task.
- #define [RF_UART_TASK_NAME](#) "RF UART task"
Name of the RF UART task.
- #define [RF_UART_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 10)
Stack size of the RF UART task.
- #define [RF_UART_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 12)
Priority of the RF UART task.
- #define [WIFI_APP_TASK_NAME](#) "WiFi app task"
Name of the WiFi app task.
- #define [WIFI_APP_STACK_SIZE](#) (configMINIMAL_STACK_SIZE * 4)
Stack size of the WiFi app task.
- #define [WIFI_APP_TASK_PRIORITY](#) (tskIDLE_PRIORITY + 8)
Priority of the WiFi app task.

8.16.1 Detailed Description

Defines for FreeRTOS tasks and priorities.

Date

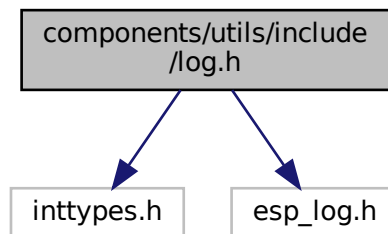
July 11, 2023

This file contains defines for FreeRTOS tasks and priorities used in the application.

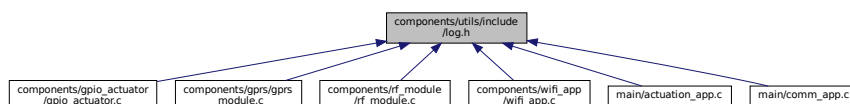
8.17 components/utils/include/log.h File Reference

Creating customizable logs.

```
#include <inttypes.h>
#include "esp_log.h"
Include dependency graph for log.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define LOG_COLOR_WHITE "37"`
ANSI color code for white.
- `#define LOG_COMM(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_CYAN) tag, __VA_ARGS__)`
Communication logs.
- `#define LOG_DATA(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_WHITE) tag, __VA_ARGS__)`
Data logs.
- `#define LOG_ACTUATION(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_PURPLE) tag, __VA_ARGS__)`
Actuation logs.
- `#define LOG_MANAGER(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_GREEN) tag, __VA_ARGS__)`
System manager logs.

8.17.1 Detailed Description

Creating customizable logs.

Date

June 15, 2022

This file contains macros for creating customizable logs.

8.17.2 Macro Definition Documentation

8.17.2.1 LOG_ACTUATION

```
#define LOG_ACTUATION(  
    tag,  
    ... ) ESP_LOGI (LOG_COLOR (LOG_COLOR_PURPLE) tag, __VA_ARGS__)
```

Actuation logs.

This macro is used to create actuation logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

8.17.2.2 LOG_COLOR_WHITE

```
#define LOG_COLOR_WHITE "37"
```

ANSI color code for white.

This define specifies the ANSI color code for white, which can be used to customize log colors.

8.17.2.3 LOG_COMM

```
#define LOG_COMM(  
    tag,  
    ... ) ESP_LOGI (LOG_COLOR (LOG_COLOR_CYAN) tag, __VA_ARGS__)
```

Communication logs.

This macro is used to create communication logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

8.17.2.4 LOG_DATA

```
#define LOG_DATA(  
    tag,  
    ... ) ESP_LOGI(LOG_COLOR(LOG_COLOR_WHITE) tag, __VA_ARGS__)
```

Data logs.

This macro is used to create data logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

8.17.2.5 LOG_MANAGER

```
#define LOG_MANAGER(  
    tag,  
    ... ) ESP_LOGI(LOG_COLOR(LOG_COLOR_GREEN) tag, __VA_ARGS__)
```

System manager logs.

This macro is used to create actuation logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

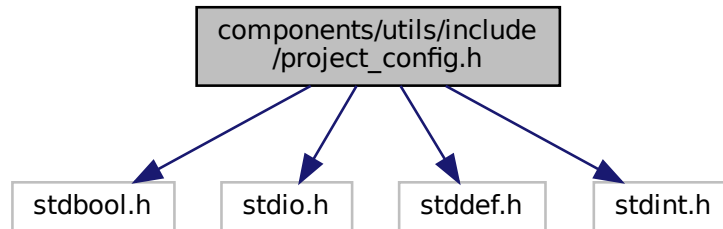
8.18 components/utils/include/project_config.h File Reference

Configuration and parameter definitions for the project.

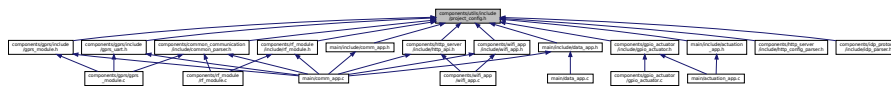
```
#include <stdbool.h>  
#include <stdio.h>  
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for project_config.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [pivot_sectors](#)
Configuration angles.

Macros

- #define **UNUSED**(x) (void)(sizeof(x))
- #define [CONFIG_SCHEDULING_MAX_VALUE](#) (10)
Maximum number of scheduling values.
- #define [CONFIG_HISTORY_MAX_VALUE](#) (20)
Maximum number of history values.
- #define [CONFIG_SECTORS_MAX_VALUE](#) (04)
Maximum number of sectors.

Typedefs

- typedef void(* [app_callback](#)) ([app_call_states](#) state, void *buffer)
Application callback function.

Enumerations

- enum `idp_type` {
`IDP_0 = 0 , IDP_1 , IDP_2 , IDP_3 ,`
`IDP_4 , IDP_5 , IDP_6 , IDP_7 ,`
`IDP_8 , IDP_9 , IDP_10 , IDP_11 ,`
`IDP_12 , IDP_13 , IDP_14 , IDP_15 ,`
`IDP_16 , IDP_INVALID = 255 }`
Enumerates different packet identifiers for communication.
- enum `pivot_states` {
`PIVOT_ON = 1 , PIVOT_OFF = 2 , PIVOT_CW = 3 , PIVOT_CCW = 4 ,`
`PIVOT_DRY = 5 , PIVOT_WET = 6 , PIVOT_PRESSURIZING = 7 , PIVOT_UNKNOWN = 0 }`
Pivot actions states.
- enum `contactor_type` { `CONTACTOR_NA = 1 , CONTACTOR_NF` }
Contactor type.
- enum `pressure_switch_type` { `PRESSURE_SWITCH_NA = 1 , PRESSURE_SWITCH_NF` }
Pressure switch type.
- enum `app_call_states` {
`CALL_LOAD_ACTION = 1 , CALL_SAVE_ACTION , CALL_READ_ACTION , CALL_SAVE_CONFIG ,`
`CALL_READ_CONFIG , CALL_SAVE_SCHEDULE_DATE , CALL_LOAD_SCHEDULE_DATE , CALL_DELETE_SCHEDULE_I`
`,`
`CALL_SAVE_SCHEDULE_ANGLE , CALL_LOAD_SCHEDULE_ANGLE , CALL_DELETE_SCHEDULE_ANGLE`
`,`
`CALL_LOAD_HISTORY ,`
`CALL_MANUAL_PIVOT , CALL_OFF_PIVOT }`
Main callback request states.

Functions

- struct `__attribute__ ((__packed__))`
Pivot actions parameters.

Variables

- `pivot_actions`
- `pivot_config`
- `network_config`
- `eco_mode_config`
- `sector_config`
- `pivot_scheduling_date`
- `pivot_scheduling_angle`
- `pivot_history`

8.18.1 Detailed Description

Configuration and parameter definitions for the project.

This file contains configuration and parameter definitions for the project.

8.18.2 Macro Definition Documentation

8.18.2.1 CONFIG_HISTORY_MAX_VALUE

```
#define CONFIG_HISTORY_MAX_VALUE (20)
```

Maximum number of history values.

This define specifies the maximum number of history values allowed in the project. It can be used to define the size of arrays or data structures related to history.

8.18.2.2 CONFIG_SCHEDULING_MAX_VALUE

```
#define CONFIG_SCHEDULING_MAX_VALUE (10)
```

Maximum number of scheduling values.

This define specifies the maximum number of scheduling values allowed in the project. It can be used to define the size of arrays or data structures related to scheduling.

8.18.2.3 CONFIG_SECTORS_MAX_VALUE

```
#define CONFIG_SECTORS_MAX_VALUE (04)
```

Maximum number of sectors.

This define specifies the maximum number of sectors allowed in the project. It can be used to define the size of arrays or data structures related to sectors.

8.18.3 Typedef Documentation

8.18.3.1 app_callback

```
typedef void(* app_callback) (app_call_states state, void *buffer)
```

Application callback function.

Function signature for the application callback function.

Parameters

<i>state</i>	The state of the main callback request.
<i>buffer</i>	The buffer containing the data associated with the request.

8.18.4 Enumeration Type Documentation

8.18.4.1 app_call_states

enum `app_call_states`

Main callback request states.

Deprecated

Enumeration defining the states for the main callback request.

Enumerator

CALL_LOAD_ACTION	Load configuration read request
CALL_SAVE_ACTION	Save configuration read request
CALL_READ_ACTION	Read configuration read request
CALL_SAVE_CONFIG	Save configuration request
CALL_READ_CONFIG	Read configuration request
CALL_SAVE_SCHEDULE_DATE	Save scheduling based on date request
CALL_LOAD_SCHEDULE_DATE	Load scheduling based on date request
CALL_DELETE_SCHEDULE_DATE	Delete scheduling based on date request
CALL_SAVE_SCHEDULE_ANGLE	Save scheduling based on angle request
CALL_LOAD_SCHEDULE_ANGLE	Load scheduling based on angle request
CALL_DELETE_SCHEDULE_ANGLE	Delete scheduling based on angle request
CALL_LOAD_HISTORY	Load history request
CALL_MANUAL_PIVOT	Manual pivot request
CALL_OFF_PIVOT	Pivot off request

8.18.4.2 contactor_type

enum `contactor_type`

Contactor type.

Enumeration defining the contactor type.

Enumerator

CONTACTOR_NA	Normally open (NO) contactor
CONTACTOR_NF	Normally closed (NC) contactor

8.18.4.3 idp_type

enum `idp_type`

Enumerates different packet identifiers for communication.

This enum defines packet identifiers used for communication. Each identifier represents a specific action or configuration parameter.

8.18.4.4 pivot_states

```
enum pivot_states
```

Pivot actions states.

Enumeration defining pivot actions states.

Enumerator

PIVOT_ON	Pivot on
PIVOT_OFF	Pivot off
PIVOT_CW	Pivot in ClockWise mode (advanced)
PIVOT_CCW	Pivot in Counter ClockWise mode (reverse)
PIVOT_DRY	Irrigation off
PIVOT_WET	Irrigation on
PIVOT_PRESSURIZING	Pivot pressurizing
PIVOT_UNKNOWN	Unknown pivot state

8.18.4.5 pressure_switch_type

```
enum pressure_switch_type
```

Pressure switch type.

Enumeration defining the pressure switch type.

Enumerator

PRESSURE_SWITCH_NA	Normally open (NO) pressure switch
PRESSURE_SWITCH_NF	Normally closed (NC) pressure switch

8.18.5 Function Documentation

8.18.5.1 __attribute__()

```
struct __attribute__ (
    (__packed__) )
```

Pivot actions parameters.

History parameters.

Scheduling angle parameters.

Scheduling date parameters.

Configuration parameters.

Structure defining pivot actions parameters.

Structure defining the configuration parameters.

Structure defining the network configuration parameters.

Structure defining the eco mode configuration parameters.

Structure defining the sectors configuration parameters.

Structure defining the scheduling parameters based on date.

Structure defining the scheduling parameters based on angle.

Structure defining the history parameters. < Power state of the pivot (PIVOT_ON or PIVOT_OFF)

< Rotation mode of the pivot (PIVOT_CW or PIVOT_CCW)

< Watering state of the pivot (PIVOT_DRY or PIVOT_WET)

< Percentage value from 0 to 100

< Pivot ID

< Contactor type

< Pressure switch type

< Pressurization time

< On/off time

< GPRS ID

< Eco mode enabled flag

< Start time

< End time

< Sector enabled flag

< Array of pivot sectors

< Scheduling ID

< Flag indicating if the scheduling is running

< Start date

< End date

< Pivot actions

< Scheduling ID

< Flag indicating if the scheduling is running

< Start date

< End angle

< Pivot actions

< Pivot actions

< Start angle

< End angle

< Start date

< End date

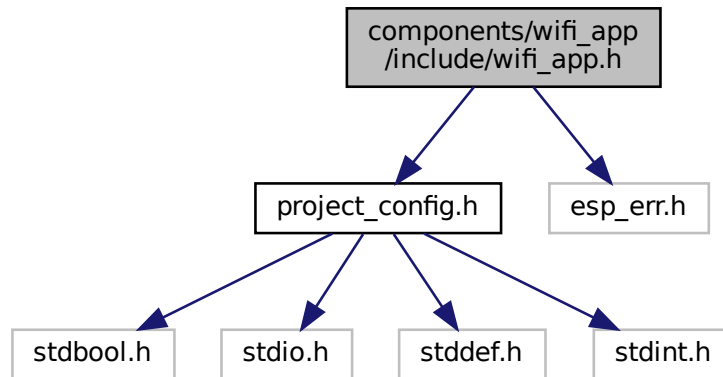
8.19 components/wifi_app/include/wifi_app.h File Reference

Wi-Fi application functions and callbacks.

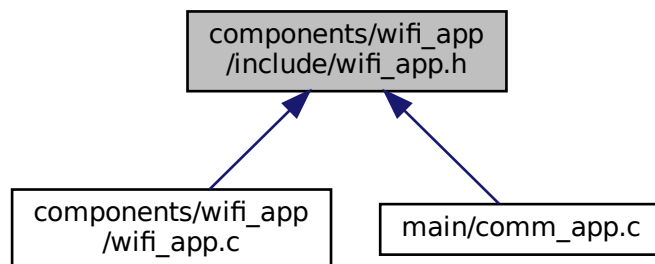
```
#include "project_config.h"
```

```
#include "esp_err.h"
```

Include dependency graph for wifi_app.h:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t` [wifi_app_init](#) (`char *wifi_ssid`)
Initialize the Wi-Fi application.
- `esp_err_t` [wifi_app_register_callback](#) (`app_callback` callback)
Register a callback function for Wi-Fi application events.

8.19.1 Detailed Description

Wi-Fi application functions and callbacks.

Date

January 29, 2023

This file contains the Wi-Fi application initialization and callback registration functions.

8.19.2 Function Documentation

8.19.2.1 `wifi_app_init()`

```
esp_err_t wifi_app_init (  
    char * wifi_ssid )
```

Initialize the Wi-Fi application.

This function initializes the Wi-Fi application with the provided SSID.

Parameters

<i>wifi_ssid</i>	The SSID of the Wi-Fi network.
------------------	--------------------------------

Returns

`esp_err_t` Returns `ESP_OK` if the initialization is successful, otherwise an error code.

8.19.2.2 `wifi_app_register_callback()`

```
esp_err_t wifi_app_register_callback (  
    app_callback callback )
```

Register a callback function for Wi-Fi application events.

This function registers a callback function to be called when a Wi-Fi application event occurs.

Parameters

<i>callback</i>	The callback function to be registered.
-----------------	---

Returns

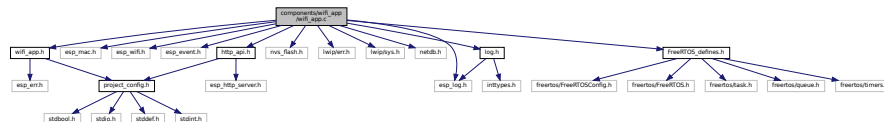
esp_err_t Returns ESP_OK if the registration is successful, otherwise an error code.

8.20 components/wifi_app/wifi_app.c File Reference

Wi-Fi application implementation.

```
#include "wifi_app.h"
#include "esp_mac.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "lwip/err.h"
#include "lwip/sys.h"
#include <netdb.h>
#include "http_api.h"
#include "FreeRTOS_defines.h"
#include "log.h"
```

Include dependency graph for wifi_app.c:



Macros

- `#define WIFI_TAG "wifi_app"`
Wi-Fi application tag for logging.
- `#define WIFI_DEFAULT_IP "192.168.0.1"`
Default IP address for the Wi-Fi access point.
- `#define WIFI_DEFAULT_MASK "255.255.255.0"`
Default subnet mask for the Wi-Fi access point.
- `#define WIFI_CHANNEL 7`
Wi-Fi channel to use for the access point.
- `#define WIFI_MAX_STA_CONN 5`
Maximum number of stations that can be connected to the access point.
- `#define WIFI_APP_SIZE_QUEUE_EVENT 5`
Size of the Wi-Fi application queue for events.

Functions

- `esp_err_t wifi_app_init (char *wifi_ssid)`
Initialize the Wi-Fi application.
- `esp_err_t wifi_app_register_callback (app_callback callback)`
Register a callback function for Wi-Fi application events.

8.20.1 Detailed Description

Wi-Fi application implementation.

Date

January 20, 2023

This file contains the implementation of the Wi-Fi application. It includes functions to start the Wi-Fi application, handle Wi-Fi events, and manage the Wi-Fi application task.

8.20.2 Function Documentation

8.20.2.1 `wifi_app_init()`

```
esp_err_t wifi_app_init (  
    char * wifi_ssid )
```

Initialize the Wi-Fi application.

This function initializes the Wi-Fi application with the provided SSID.

Parameters

<code>wifi_ssid</code>	The SSID of the Wi-Fi network.
------------------------	--------------------------------

Returns

`esp_err_t` Returns `ESP_OK` if the initialization is successful, otherwise an error code.

8.20.2.2 `wifi_app_register_callback()`

```
esp_err_t wifi_app_register_callback (  
    app_callback callback )
```

Register a callback function for Wi-Fi application events.

This function registers a callback function to be called when a Wi-Fi application event occurs.

Parameters

<code>callback</code>	The callback function to be registered.
-----------------------	---

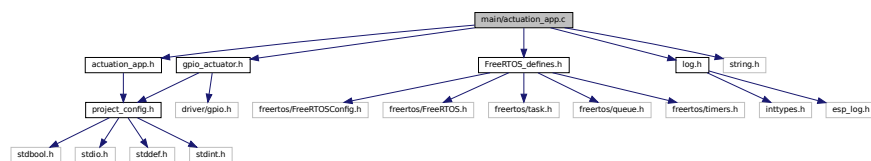
Returns

esp_err_t Returns ESP_OK if the registration is successful, otherwise an error code.

8.21 main/actuation_app.c File Reference

actuation control class

```
#include "actuation_app.h"
#include "gpio_actuator.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include <string.h>
Include dependency graph for actuation_app.c:
```



Macros

- #define **ACTUATION_APP_TAG** "actuation_app"
- #define **ACTUATION_APP_POWER_TIME** 10000
- #define **ACTUATION_APP_WATERING_TIME** 30000
- #define **ACTUATION_APP_ROTATION_TIME** 3000
- #define **ACTUATION_APP_PERCENTIMETER_TIME** 61000

Functions

- void **actuation_app_task** (void *arg)
Task responsible for monitoring possible changes in equipment status.
- bool **actuation_app_init** (const app_callback callback)
starts the tasks of reading and actuation of the I/O ports
- void **actuation_app_set_actions** (const pivot_actions config_in, bool alert_change)
apply the configuration to the drives
- void **actuation_app_get_actions** (pivot_actions *config_out, size_t config_size)
read current status on IO ports
- void **actuation_app_set_pump** (bool pump_state)
- void **actuation_app_shutdown** (void)

8.21.1 Detailed Description

actuation control class

Date

June 23, 2022

8.21.2 Function Documentation

8.21.2.1 `actuation_app_get_actions()`

```
void actuation_app_get_actions (
    pivot_actions * config_out,
    size_t config_size )
```

read current status on IO ports

Parameters

<i>config_out</i>	- [out]: pivot configuration structure
<i>config_length</i>	- [out]: structure size

8.21.2.2 `actuation_app_init()`

```
bool actuation_app_init (
    const app_callback callback )
```

starts the tasks of reading and actuation of the I/O ports

Parameters

<i>app_callback</i>	- [in]: function used with return to main application class
---------------------	---

Returns

- true: success
- false: fail to initialize

Here is the call graph for this function:



8.21.2.3 actuation_app_set_actions()

```
void actuation_app_set_actions (
    const pivot_actions config_in,
    bool alert_change )
```

apply the configuration to the drives

Parameters

<code>config_in</code>	- [in]: function used with return to main application class
------------------------	---

8.21.2.4 actuation_app_task()

```
void actuation_app_task (
    void * arg )
```

Task responsible for monitoring possible changes in equipment status.

Parameters

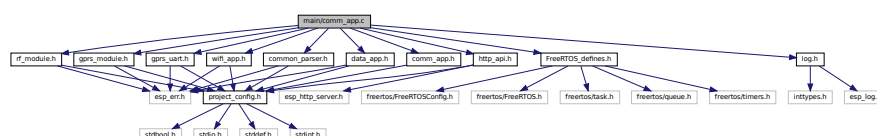
<code>arg</code>	- [in]: task argument (default NULL)
------------------	--------------------------------------

8.22 main/comm_app.c File Reference

communication class control application

```
#include "comm_app.h"
#include "rf_module.h"
#include "gprs_module.h"
#include "gprs_uart.h"
#include "wifi_app.h"
#include "http_api.h"
#include "common_parser.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include "data_app.h"
```

Include dependency graph for comm_app.c:



Data Structures

- struct [comm_app_request](#)

Macros

- #define **COMM_APP_TAG** "comm_app"
- #define **COMM_APP_SIZE_QUEUE** 10

Functions

- void [comm_app_task](#) (void *arg)
communication reception task
- bool [comm_app_init](#) (const [app_callback](#) callback)
starts all communication modules
- uint16_t [comm_app_get_degree](#) (void)
starts all communication modules
- void [comm_app_send_event](#) (pivot_actions pivot_status)
send events over communication interfaces
- void **comm_app_set_config** (const pivot_config config)
- void **comm_app_set_actions** (const pivot_actions action, const pivot_config config, uint16_t start_angle, uint16_t end_angle)
- void **comm_app_send_actions** (void)
- void **comm_app_send_idp_resp** ([idp_type](#) idp, char *pivo_id, char *scheduling_id)
- void **MODULES_NOTIFY_APP** (void *notify_buffer)
- void [RF_MODULE_NOTIFY_APP](#) (void *notify_buffer)
trigger callback to each status message received (write and read)
- void [GPRS_MODULE_NOTIFY_APP](#) (void *notify_buffer)
Notifies the application about status messages.

8.22.1 Detailed Description

communication class control application

Date

June 23, 2022

8.22.2 Function Documentation

8.22.2.1 GPRS_MODULE_NOTIFY_APP()

```
void GPRS_MODULE_NOTIFY_APP (
    void * notify_buffer )
```

Notifies the application about status messages.

This function triggers a callback for each status message received (write and read).

Parameters

<code>notify_buffer</code>	Pointer to the notification buffer.
----------------------------	-------------------------------------

8.22.2.2 RF_MODULE_NOTIFY_APP()

```
void RF_MODULE_NOTIFY_APP (
    void * notify_buffer )
```

trigger callback to each status message received (write and read)

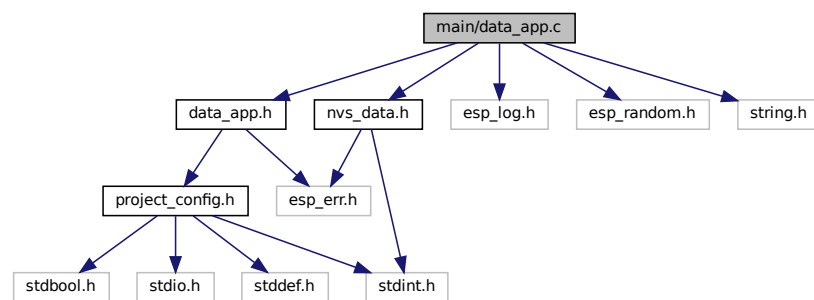
Parameters

<code>config_in[in]</code>	configuration structure
----------------------------	-------------------------

8.23 main/data_app.c File Reference

memory control application

```
#include "data_app.h"
#include "nvs_data.h"
#include "esp_log.h"
#include "esp_random.h"
#include <string.h>
Include dependency graph for data_app.c:
```



Macros

- `#define DATA_APP_TAG "data_app"`
- `#define DATA_APP_LABEL_ACTION "space_action"`
- `#define DATA_APP_LABEL_CONFIG "space_config"`
- `#define DATA_APP_LABEL_SCHEDULING_DATE "space_s_date"`

- #define **DATA_APP_LABEL_SCHEDULING_ANGLE** "space_s_angle"
- #define **DATA_APP_LABEL_HISTORY** "space_history"
- #define **DATA_APP_LABEL_TIMESTAMP** "space_timestamp"
- #define **DATA_KEY_ACTIONS** "label_actions"
- #define **DATA_KEY_CONFIG** "label_config"
- #define **DATA_KEY_SCHEDULING_DATE** "label_s_date"
- #define **DATA_KEY_SCHEDULING_ANGLE** "label_s_angle"
- #define **DATA_KEY_HISTORY** "label_history"
- #define **DATA_KEY_TIMESTAMP** "label_timestamp"

Functions

- esp_err_t **data_app_init** (void)
start all memory modules
- esp_err_t **data_app_save_actions** (const pivot_actions *action, size_t size)
sends the queue a request to save a new action
- esp_err_t **data_app_load_actions** (pivot_actions *out_action, size_t size)
sends the queue a request to load a current action.
- esp_err_t **data_app_save_config** (const pivot_config *config, size_t size)
sends the queue a request to save a new configuration
- esp_err_t **data_app_load_config** (pivot_config *out_config, size_t size)
sends the queue a request to load a configuration.
- void **data_app_gen_scheduling_key** (char *scheduling_id)
- esp_err_t **data_app_save_scheduling** (data_scheduling_type scheduling_type, const void *value, size_t size)
- esp_err_t **data_app_load_scheduling** (data_scheduling_type scheduling_type, void *out_value, size_t size)
- esp_err_t **data_app_delete_scheduling** (data_scheduling_type scheduling_type, char *scheduling_id)
- esp_err_t **data_app_save_new_history** (pivot_history new_history)
- esp_err_t **data_app_save_old_history** (time_t end_date, uint16_t end_angle)
- esp_err_t **data_app_load_history** (pivot_history *out_history, size_t size)
- esp_err_t **data_app_save_timestamp** (time_t *timestamp)
- esp_err_t **data_app_load_timestamp** (time_t *out_timestamp)
- size_t **data_app_get_data_size** (const char *label_name, const char *key)
Gets the data size for the informed key.

8.23.1 Detailed Description

memory control application

Date

Set 18, 2022

8.23.2 Macro Definition Documentation

8.23.2.1 DATA_APP_LABEL_ACTION

```
#define DATA_APP_LABEL_ACTION "space_action"
```

NVS access space

8.23.2.2 DATA_APP_TAG

```
#define DATA_APP_TAG "data_app"
```

class log tag

8.23.2.3 DATA_KEY_ACTIONS

```
#define DATA_KEY_ACTIONS "label_actions"
```

NVS access label

8.23.3 Function Documentation

8.23.3.1 data_app_get_data_size()

```
size_t data_app_get_data_size (  
    const char * label_name,  
    const char * key )
```

Gets the data size for the informed key.

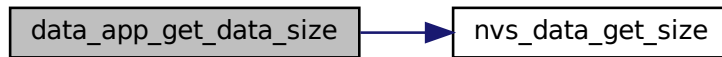
Parameters

<i>key</i>	- access key defined in project_config.h
------------	--

Returns

- 0: in case of errors
- Data size.

Here is the call graph for this function:

**8.23.3.2 data_app_init()**

```
esp_err_t data_app_init (
    void )
```

start all memory modules

Returns

- ESP_OK: success
- ESP_FAIL: fail to initialize

8.23.3.3 data_app_load_actions()

```
esp_err_t data_app_load_actions (
    pivot_actions * out_action,
    size_t size )
```

sends the queue a request to load a current action.

Parameters

<i>out_value</i>	- [in]: return pointer with the value read from the label.
<i>size</i>	- [in]: size allocated for pointer(out_value).

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.23.3.4 data_app_load_config()

```
esp_err_t data_app_load_config (
    pivot_config * out_config,
    size_t size )
```

sends the queue a request to load a configuration.

Parameters

<i>out_value</i>	- [in]: return pointer with the value read from the label.
<i>size</i>	- [in]: size allocated for pointer(out_value).

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.23.3.5 data_app_save_actions()

```
esp_err_t data_app_save_actions (
    const pivot_actions * action,
    size_t size )
```

sends the queue a request to save a new action

Parameters

<i>value</i>	- [in]: content you want to save in memory.
<i>size</i>	- [in]: size of content

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.23.3.6 data_app_save_config()

```
esp_err_t data_app_save_config (
    const pivot_config * config,
    size_t size )
```

sends the queue a request to save a new configuration

Parameters

<i>value</i>	- [in]: content you want to save in memory.
<i>size</i>	- [in]: size of content

Returns

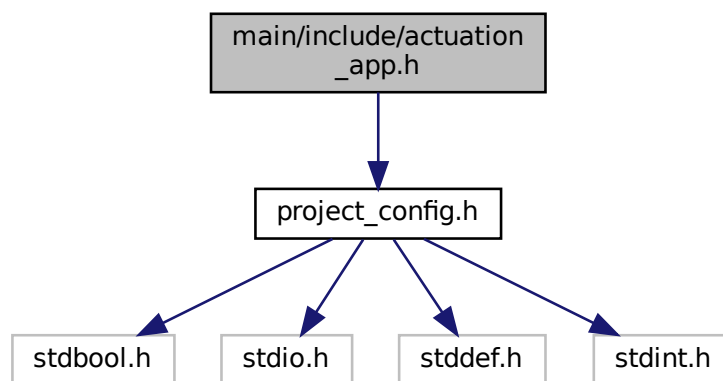
- ESP_OK: success
- ESP_FAIL: fail to save

8.24 main/include/actuation_app.h File Reference

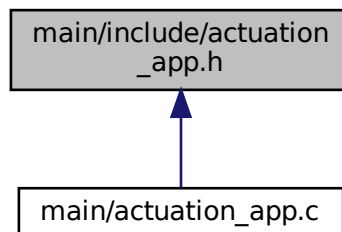
actuation control class

```
#include "project_config.h"
```

Include dependency graph for actuation_app.h:



This graph shows which files directly or indirectly include this file:



Functions

- bool `actuation_app_init` (const `app_callback` callback)
starts the tasks of reading and actuation of the I/O ports
- void `actuation_app_set_actions` (const `pivot_actions` `config_in`, bool `alert_change`)
apply the configuration to the drives
- void `actuation_app_get_actions` (`pivot_actions` *`config_out`, `size_t` `config_size`)
read current status on IO ports
- void `actuation_app_set_pump` (bool `pump_state`)
- void `actuation_app_shutdown` (void)

8.24.1 Detailed Description

actuation control class

Date

June 23, 2022

8.24.2 Function Documentation

8.24.2.1 `actuation_app_get_actions()`

```
void actuation_app_get_actions (
    pivot_actions * config_out,
    size_t config_size )
```

read current status on IO ports

Parameters

<code>config_out</code>	- [out]: pivot configuration structure
<code>config_length</code>	- [out]: structure size

8.24.2.2 `actuation_app_init()`

```
bool actuation_app_init (
    const app_callback callback )
```

starts the tasks of reading and actuation of the I/O ports

Parameters

<code>app_callback</code>	- [in]: function used with return to main application class
---------------------------	---

Returns

- true: success
- false: fail to initialize

Here is the call graph for this function:

**8.24.2.3 actuation_app_set_actions()**

```
void actuation_app_set_actions (
    const pivot_actions config_in,
    bool alert_change )
```

apply the configuration to the drives

Parameters

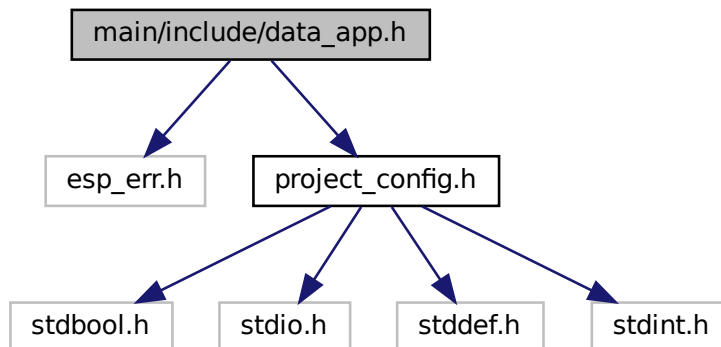
<code>config_in</code>	- [in]: function used with return to main application class
------------------------	---

8.25 main/include/data_app.h File Reference

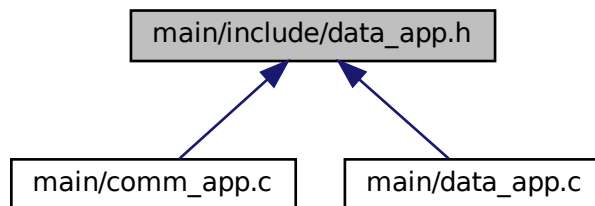
memory control application

```
#include "esp_err.h"
#include "project_config.h"
```


Include dependency graph for data_app.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `data_scheduling_type` { `data_scheduling_date` = 0 , `data_scheduling_angle` }

Functions

- `esp_err_t data_app_init` (void)
start all memory modules
- `esp_err_t data_app_save_actions` (const `pivot_actions` *action, `size_t` size)
sends the queue a request to save a new action
- `esp_err_t data_app_load_actions` (`pivot_actions` *out_action, `size_t` size)
sends the queue a request to load a current action.
- `esp_err_t data_app_save_config` (const `pivot_config` *config, `size_t` size)
sends the queue a request to save a new configuration
- `esp_err_t data_app_load_config` (`pivot_config` *out_config, `size_t` size)

sends the queue a request to load a configuration.

- void **data_app_gen_scheduling_key** (char *scheduling_id)
- esp_err_t **data_app_save_scheduling** (data_scheduling_type scheduling_type, const void *value, size_t size)
- esp_err_t **data_app_load_scheduling** (data_scheduling_type scheduling_type, void *out_value, size_t size)
- esp_err_t **data_app_delete_scheduling** (data_scheduling_type scheduling_type, char *scheduling_id)
- esp_err_t **data_app_save_new_history** (pivot_history new_history)
- esp_err_t **data_app_save_old_history** (time_t end_date, uint16_t end_angle)
- esp_err_t **data_app_load_history** (pivot_history *out_history, size_t size)
- esp_err_t **data_app_save_timestamp** (time_t *timestamp)
- esp_err_t **data_app_load_timestamp** (time_t *out_timestamp)
- size_t **data_app_get_data_size** (const char *label_name, const char *key)

Gets the data size for the informed key.

8.25.1 Detailed Description

memory control application

Date

June 15, 2022

8.25.2 Function Documentation

8.25.2.1 data_app_get_data_size()

```
size_t data_app_get_data_size (
    const char * label_name,
    const char * key )
```

Gets the data size for the informed key.

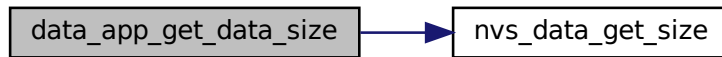
Parameters

<i>key</i>	- access key defined in project_config.h
------------	--

Returns

- 0: in case of errors
- Data size.

Here is the call graph for this function:

**8.25.2.2 data_app_init()**

```
esp_err_t data_app_init (  
    void )
```

start all memory modules

Returns

- ESP_OK: success
- ESP_FAIL: fail to initialize

8.25.2.3 data_app_load_actions()

```
esp_err_t data_app_load_actions (  
    pivot_actions * out_action,  
    size_t size )
```

sends the queue a request to load a current action.

Parameters

<i>out_value</i>	- [in]: return pointer with the value read from the label.
<i>size</i>	- [in]: size allocated for pointer(out_value).

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.25.2.4 data_app_load_config()

```
esp_err_t data_app_load_config (
    pivot_config * out_config,
    size_t size )
```

sends the queue a request to load a configuration.

Parameters

<i>out_value</i>	- [in]: return pointer with the value read from the label.
<i>size</i>	- [in]: size allocated for pointer(out_value).

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.25.2.5 data_app_save_actions()

```
esp_err_t data_app_save_actions (
    const pivot_actions * action,
    size_t size )
```

sends the queue a request to save a new action

Parameters

<i>value</i>	- [in]: content you want to save in memory.
<i>size</i>	- [in]: size of content

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

8.25.2.6 data_app_save_config()

```
esp_err_t data_app_save_config (
    const pivot_config * config,
    size_t size )
```

sends the queue a request to save a new configuration

Parameters

<i>value</i>	- [in]: content you want to save in memory.
<i>size</i>	- [in]: size of content

Returns

- ESP_OK: success
- ESP_FAIL: fail to save

